

SynSmith: Adversarial Multi-Critic Prompt Debugging for Synthetic Data Generation

Alexander Apartsin¹ Yehudit Aperstein²

¹Holon Institute of Technology, Israel ²Afeka College of Engineering, Israel

Abstract

Large language models are increasingly used to generate labeled training data when real labels are scarce, but the synthesis suffers from three pathologies: attribute infidelity (generated text does not match the requested label), over-polished realism (samples are detectably synthetic), and shallow diversity (samples paraphrase a small set of templates). We present **SynSmith**, a seven-critic prompt-rewriting loop combining three baseline critics (attribute verifier, realism discriminator, diversity auditor) with four GAN-inspired mode-collapse adversaries (a Pack Discriminator, a Mode-Seeking ratio, a Mode Hunter with persistent banned-phrasings memory, and a density-ratio Coverage Hole Finder). Under a structured-feedback contract, each critic emits named, typed complaints that the prompt updater satisfies jointly under a length budget. Synth-only training on SynSmith's batch reaches [0.91,1.07] of real-only macro F1 across three benchmarks (SST-2, Banking77, TREC) at 10 to 30 real-train examples per class, where 1.0 matches real-only and the bracket is the 95% confidence interval over five seeds (Figure 2, Table 4). A per-component leave-one-out attributes the gain to iteration under the structured-feedback contract, the Pack Discriminator, and the Mode Hunter. A post-hoc adversary audit with a real-vs-real null reference produces apples-to-apples scores across all conditions and is transferable to any multi-critic synthetic-data system. A reference implementation is available at github.com/ApartsinProjects/SynSmith.

1 Introduction

Large language models are widely used to generate labeled training data when real data is scarce. Naive prompting produces three intertwined pathologies: *attribute infidelity* (the text does not reflect the requested labels), *over-polished realism* (samples are detectably synthetic), and *shallow diversity* (samples are paraphrases of a small set of templates). A common response is to add critics: an attribute verifier, a realism discriminator, and a diversity auditor. The literature on automatic prompt optimization with LLM judges [14][15][16][17][18][19][25] reads as a steady accumulation: more critics, more rewriting capacity, better data. We ask whether that accumulation actually improves downstream utility, and how the field would tell.

Our angle is structural. The GAN literature spent a decade naming the same three pathologies in image generators, and the defenses transferred include PacGAN [1] for in-batch mode collapse, MSGAN [3] for mode-seeking variance, minibatch features [2], unrolled GANs [7], WGAN [8] for distributional matching, and density-ratio estimation [9] for coverage. We adapt four of these defenses into a multi-critic prompt-debugging loop and ask whether the result produces synthetic data whose decision-boundary footprint complements other generation regimes, an empirical question a single-classifier macro-F1 protocol cannot answer. The evaluation lens

that does answer it, cross-condition classifier ensembling, is also the source of the headline result, and is described separately under contribution (C1) below.

Contributions. Three.

(C1) SynSmith, a single seven-critic prompt-debugging loop. Three baseline critics (attribute verifier, realism discriminator, diversity auditor; Section 4) run alongside four GAN-inspired adversaries adapted to the prompt-debugging loop: a Pack Discriminator (PacGAN [1]), a Mode-Seeking ratio (MSGAN [3]), a Mode Hunter with a persistent banned-phrasings memory, and a Coverage Hole Finder via density-ratio estimation [9] (Section 5). SynSmith is the named, recommended configuration with all components ON; the empirical headline is that synth-only training on SynSmith's batch reaches a relative-ratio bracket of [0.91,1.07] of real-only macro F1 across three standard benchmarks at 10 to 30 real-train per class, where 1.0 matches real-only. Per-dataset: SST-2 1.04 [1.00,1.07] (beats real-only), Banking77 0.92 [0.91,0.93] (trails by 7 to 9% of real-only), TREC 1.00 [0.92,1.08] (matches real-only). Section 7.5 carries the per-dataset table. On customer-support intent classification, SynSmith additionally combines with the simplest iterated baseline through cross-condition classifier ensembling to reach macro F1 0.947 ± 0.056 (Section 7.3).

(C2) An ablation that attributes the gain to specific components, plus a post-hoc adversary audit. A leave-one-out ablation (Section 7.3, Appendix H) drops each component and reports the F1 delta: iteration with the structured-feedback contract carries the largest single share, the Pack Discriminator and Mode Hunter contribute distinct measurable per-class lift on the failure modes they were designed to attack, and Mode-Seeking and Coverage Hole Finder do not measurably differentiate conditions in our experiments. The post-hoc adversary audit (Section 8) runs every critic on every condition's final batch with re-seeded RNG and a real-vs-real null reference, retiring the tautological "n/a" entries of single-critic ablation tables. Together they pin SynSmith's lift to specific named mechanisms rather than to "we used many critics".

(C3) A structured-feedback contract for adversarial multi-critic loops. Each critic emits named, typed complaints rather than scalar rewards; the prompt updater is required to satisfy them jointly under a length budget (Section 4). The contract yields two properties absent from scalar-reward stacks: locally targeted rewrites (each named complaint admits a specific rewrite clause) and joint constraint satisfaction (no single critic dominates by being the loudest). A reference implementation accompanies the paper at github.com/ApartsinProjects/SynSmith.

Operating regime. SynSmith is designed for the scarce-real regime: 3 to 50 real examples per class is enough to anchor the empirical-referent Verifier (Section 4.3), the few-shot generator pool, and the Realism, Coverage Hole, and Pack discriminators. Below 3 real examples per class the per-class anchoring degrades to schema-name-only verification; above ~ 100 per class the synth-vs-real gap a synth-only classifier can close shrinks because the real-only baseline is already strong. The reported runs sit inside this band: 30 real-train examples per class on SST-2 (binary), 30 per class on Banking77 (10 fine-grained card-payment intents), 10 per class on TREC (6 coarse question types), and 6 per class on customer-support (5 intents). Section 6.5 lists each critic's real-data dependence in detail and Section 7's headline tables report real-per-class alongside synth-only accuracy so the regime is unambiguous.

2 Related work

2.1 Synthetic data generation with LLMs

LLM-based data augmentation strategies range from zero-shot generation prompts to multi-step pipelines that combine retrieval, filtering, and self-correction. Self-Instruct [5] and Unnatural Instructions [4] bootstrap instruction-tuning corpora through LLM self-generation. InPars [10], SuperGen [11], and ProGen [12] use LLMs to synthesize training data for downstream classifiers and rerankers. West et al. (Symbolic Knowledge Distillation) [13] use a large LLM to distill structured knowledge into a smaller model.

The sharpest single comparison point is AttrPrompt [49], which conditions one-shot LLM generation on diverse attribute vectors and reports diversity and downstream-accuracy gains over class-conditional prompting on NYT, Amazon, Reddit, and arXiv. AttrPrompt establishes attribute-conditioned generation as a principle but is a single-shot generator with no iterative critic loop; the four GAN-style adversaries that drive SynSmith (Pack Discriminator, Mode-Seeking, Mode Hunter, Coverage Hole Finder) have no analogue in AttrPrompt. WANLI [50] pairs dataset cartography with GPT-3 generation and human relabeling, demonstrating that distribution-aware sampling produces more challenging synthetic NLI than uniform sampling; the "real-distribution anchor" idea is shared, but WANLI requires a human relabel step that SynSmith replaces with an LLM Verifier whose `attribute_match` is gated by the class label only (Section 4).

Iterative-feedback synth generators are the next-closest family. S3 [51] trains a small classifier on the synthetic batch, extrapolates its errors on a gold validation slice, and re-prompts the LLM with those errors as targets; ProGen [12] uses noise-tolerant influence functions on a similar small-model loop. TarGEN [52] adds a self-correction module for label noise after multi-step prompting. ARISE [53] iterates rule induction over syntactic n-grams and bootstrapped filtering. PACE / Synthline [42] combines multi-sample prompting with an actor-critic prompt-editing loop and reports F1 gains of 6 to 43.8 percentage points on Requirements Engineering classification, while naming the "diversity-as-metric vs utility-as-goal" tension that a single-objective optimizer faces. Genetic Prompt [43] treats class-conditional attributes as gene sequences and uses the LLM to simulate crossover and mutation. CoT-Self-Instruct [46] applies Answer-Consistency and Rejecting-Instruction-Preferences filters in the reasoning domain. Self-Refine [54] and Reflexion [55] generalize iterative critique-refine loops to general agent tasks but operate on single outputs, not data distributions, and use a single critic role.

All of these methods optimize one signal at a time. S3 and ProGen optimize for downstream validation accuracy; TarGEN for label correctness; ARISE for rule satisfaction; PACE for lexical diversity; AttrPrompt for attribute coverage at generation time. SynSmith runs a multi-critic stack in which seven critics (three baseline plus four GAN-style adversaries) each emit named, falsifiable complaints, and the prompt updater is asked to satisfy them jointly. The critic-coverage matrix below makes the structural difference explicit:

Table 1: Critic-coverage matrix. One column per critic family (rows are prior methods). SynSmith is the only row that combines all five families: attribute fidelity, real-vs-synthetic realism, mode-collapse defense (the four GAN-style adversaries), coverage-gap repair, and an iterated multi-critic updater. AttrPrompt establishes attribute conditioning but is one-shot; S3, ProGen, TarGEN, ARISE, and PACE all iterate but optimize a single objective; Reflexion / Self-Refine / DSPy / TextGrad optimize single outputs or single prompts, not data distributions. Full critic-by-critic matrix is in Appendix A.

Method	Attribute fidelity	Realism (real-vs-synth)	Mode-collapse defense	Coverage gaps	Iterated multi-critic
SynSmith	✓	✓	✓ (4 axes)	✓	✓
AttrPrompt	✓	–	–	–	– (1-shot)
S3 / ProGen	–	implicit	–	implicit	single-objective
TarGEN / ARISE	label / rule	–	–	–	single-objective
PACE / Synthline	–	–	–	–	actor-critic
Reflexion / Self-Refine / DSPy / TextGrad	metric	–	–	–	single output / prompt

PerFine [45] is the closest 2025 analog of our "named complaints, not scalar rewards" feedback contract: it emits structured complaints along four named dimensions (tone, vocabulary, sentence structure, topicality) and uses a cross-iteration knockout. PerFine reports +7.8 to +13.4 percentage points G-Eval over baselines on Yelp / Goodreads / Amazon, evidence that structured multi-dimensional critique outperforms scalar judgments outside of synthetic data generation; our seven-critic stack extends the same contract to four GAN-style adversaries.

SynSmith's positioning, in one sentence: *the first attribute-conditioned synthetic-data framework with a multi-critic adversarial loop in which every critic is anchored to the real-seed distribution* (via empirical-anchor calibration of the Verifier, domain-canonical veto of the Mode Hunter, and distribution-aware judging of the Realism Discriminator), evaluated across four task families (sentiment, fine-grained intent, question-type, NLI) that no single competitor's headline tables cover.

2.2 Prompt optimization via search and rewrite

Several recent works treat prompt design as a discrete search problem. APE [14] generates many candidate prompts and selects by held-out score. OPRO [15] uses LLMs as optimizers that propose new candidates conditioned on the trajectory so far. EvoPrompt [16] applies evolutionary search to prompts. PromptBreeder [17] uses LLM-driven mutation operators within an evolutionary loop. DSPy [18] compiles modular programs with optimized prompts. TextGrad [19] treats prompts and outputs as differentiable artifacts updated by LLM-generated "text gradients". More recent work in 2025 includes GEPA [25], which uses reflective prompt evolution with Pareto-front selection and reports gains over RL baselines, and Pareto Prompt Optimization [26], which explicitly explores the Pareto front of prompts using dominance relations rather than scalarizing objectives. The Auto-Prompt Ensemble work [27] shows a confidence-aware ensemble of optimized prompts beats single-prompt baselines on LLM-as-judge tasks.

SynSmith differs from this line in three ways. First, our objective is structurally multi-axis: seven critics (three baseline plus four mode-collapse adversaries) each emit named, falsifiable complaints rather than scalar rewards. Second, the updater is asked to satisfy them jointly under

a length budget rather than through scalarization. Third, we explicitly transfer batch-level defenses from the GAN literature, which to our knowledge no prior prompt-optimization work has done.

2.3 Multi-critic and debate-style judges

Hu et al.'s Multi-Agent Debate for LLM Judges [29] formally proves that debate amplifies judge correctness over static ensembles and introduces adaptive Beta-Binomial stopping rules. This is complementary to our work: debate adds deliberation between judges, while we add adversarial batch-level dynamics. Our seven critics are intentionally independent rather than deliberative; combining the two paradigms is a natural extension.

2.4 LLM-as-judge

LLM-as-judge methods [6] have become standard for open-ended evaluation, with G-Eval [20] systematizing rubric-driven evaluation and self-consistency [21] reducing variance. We extend that pattern to synthetic-data generation and add the specific structural property that each critic's output grammar is bounded so the updater receives *named* complaints rather than free-form prose.

2.5 Mode collapse in GANs and its measurement

Mode collapse is a long-studied failure of generative-adversarial training where the generator finds a small set of points that fool the discriminator and emits them repeatedly. Documented defenses include minibatch discrimination [2], unrolled GANs [7], mode-seeking GAN [3], Pac-GAN [1], and Wasserstein-style losses [8]. Quantification of collapse uses FID [22], precision and recall for distributions [23], and density/coverage [24]. We adapt four of these defenses into a prompt-debugging setting where the generator's parameters are frozen and the optimized object is the prompt.

2.6 Mode collapse in LLM outputs

Mode collapse has been documented for LLM outputs themselves, distinct from GAN-induced collapse. Verbalized Sampling [30] attributes the failure to "typicality bias in preference data" and proposes a training-free prompting recipe (asking the model to verbalize a probability distribution over responses) that recovers 1.6 to 2.1x more diversity on creative-writing tasks. NanoFlux [31] is the closest existing GAN-style framework for LLM data: it pairs an attacker LLM and a defender LLM under a tool-augmented judge to generate targeted training examples. Our work differs from NanoFlux in two ways: we use multiple critics rather than a single judge, and our adversaries are structural transfers from specific GAN defenses rather than free-form attackers.

2.7 Model collapse from training on synthetic data

A separate but related thread documents that training LLMs on synthetic data produced by earlier LLMs degrades the model distribution over generations. Strong Model Collapse [32] proves the failure persists even with arbitrarily small synthetic fractions. Kazdan et al.'s analysis of Shumailov 2024 [33] shows that the magnitude is sensitive to data-mixing strategy, and that accumulating (rather than replacing) real data prevents the collapse. Synthetic Eggs in Many Baskets [34] empirically connects synthetic-data diversity to downstream fine-tuning performance,

showing that more diverse synthetic sources measurably mitigate collapse. Machine-generated text detection as a pipeline filter [35] is another defense thread that connects to our Coverage Hole Finder’s density-ratio classifier.

2.8 Density-ratio estimation for synthetic-data quality

Density ratio estimation [9] recovers $p_{\text{real}}(x)/(p_{\text{real}}(x) + p_{\text{synth}}(x))$ with a binary classifier; we use it in Section 5.4 to surface real exemplars the synthetic distribution fails to cover and inject them as few-shot anchors. An ICLR 2025 SynthData-workshop paper [36] aggregates multiple density-ratio estimators with diverse hyperparameter settings to produce global and local quality scores, providing a direct technical antecedent for our Coverage Hole Finder.

2.9 Embedding-based diversity metrics for text

The Scendi Score [37] decomposes embedding-based diversity into prompt-driven and intrinsic-model components via a Schur-complement decomposition of CLIP-embedding kernels; this is the closest 2024 alternative to distinct-n and FID for text diversity. The conditional Vendi score [44] formalizes the same decomposition information-theoretically as $H(X) = H(X|T) + I(X;T)$, separating model-intrinsic diversity from prompt-driven diversity. We use a Scendi-style Schur-complement decomposition (Section 7.2) and find that the $2 \times$ iterated-vs-non-iterated Vendi gain is approximately fully intrinsic-model-driven (the prompt-residual is near zero), which is the kind of mechanistic claim the Ospanov et al. formalization makes precise. Our Mode-Seeking ratio occupies a related niche but is conditioned on attribute-vector distance rather than prompt identity.

2.10 Annotation-free synthetic-data evaluation and structural surveys

Two evaluation-side lines inform our protocol. SynQuE [47] proposes annotation-free quality estimators (Mean Distance to Medoid, MMD, Proxy-A-Distance, LLM-evaluated Lens) and reports Spearman correlation 0.38 to 0.68 with downstream classifier performance across a 32-dataset sentiment suite, complementing our supervised macro / worst-class F1 protocol. The QDC framework of Havrilla et al. [48] argues that Quality drives in-distribution generalization, Diversity drives OOD generalization, and Complexity helps both; our seven critics map onto this decomposition (the verifier and realism discriminator on the Q axis, the diversity auditor and four GAN-style adversaries on the D axis).

3 Problem formulation

Given the inputs:

- a small real example set $R = \{x_i^r\}_{i=1}^{N_r}$ ($N_r = 40$ in our experiments, 30 train and 10 held out for downstream evaluation),
- an attribute schema $\mathcal{A} = (a_1, \dots, a_K)$ where each a_k has a discrete value set V_k , and a set of invalid combinations $\mathcal{C}$,
- an initial generation prompt P_0 ,
- a generator LLM G ,
- a set of LLM-based critics $\{V, D, C\}$ and a prompt updater U .

At round t the system selects target attribute vectors $\mathbf{z}_t \in \prod_k V_k$, generates samples $S_t = G(P_t, \mathbf{z}_t)$, runs the critics, and updates the prompt:

$$P_{t+1} = U(P_t, V(S_t, \mathbf{z}_t), D(R \cup S_t), C(S_t)).$$

The optimization is multi-objective. We track three primary objective scalars:

$$\begin{aligned} \mathcal{O}_{\text{fid}}(S_t) &= \frac{1}{|S_t|} \sum_{s \in S_t} \mathbf{1}\{V(s, \mathbf{z}(s)) = \text{match}\}, \\ \mathcal{O}_{\text{real}}(S_t) &= 1 - \text{acc}(D, R \cup S_t), \\ \mathcal{O}_{\text{cov}}(S_t) &= 1/|\mathcal{A}| \sum_k H(\{\mathbf{z}_k\})/\log|V_k|, \end{aligned}$$

where $\text{acc}(D, \cdot)$ is the discriminator's accuracy on the shuffled mixed batch (chance level 0.5) and $H(\cdot)$ is the Shannon entropy of observed attribute values. The near-duplicate rate $\tau(\{x_s\}) = 1/|S| |\{x: \max_{x' \neq x} \cos(\phi(x), \phi(x')) \geq \theta\}|$ and the combination coverage are reported separately, not folded into a single weighted scalar, so each axis can be inspected independently.

In Section 5 we add four batch-level objectives that the GAN-style adversaries optimize: Pack accuracy ($\mathcal{O}_{\text{pack}}$, target 0.5), Mode-Seeking ratio relative to real ($\mathcal{O}_{\text{ms}}$, target 1.0), Mode Hunter library survivors ($\mathcal{O}_{\text{hunt}}$, target 0), and Coverage Classifier AUROC ($\mathcal{O}_{\text{auroc}}$, target 0.5).

Why optimize the prompt rather than weights? Optimizing the prompt yields four practical advantages: zero parameter footprint, inspectable optimization trajectory (every prompt version is a reviewable text artifact), immediate applicability to closed-weights models, and cheap multi-seed re-runs without GPU.

4 Method: the SynSmith loop

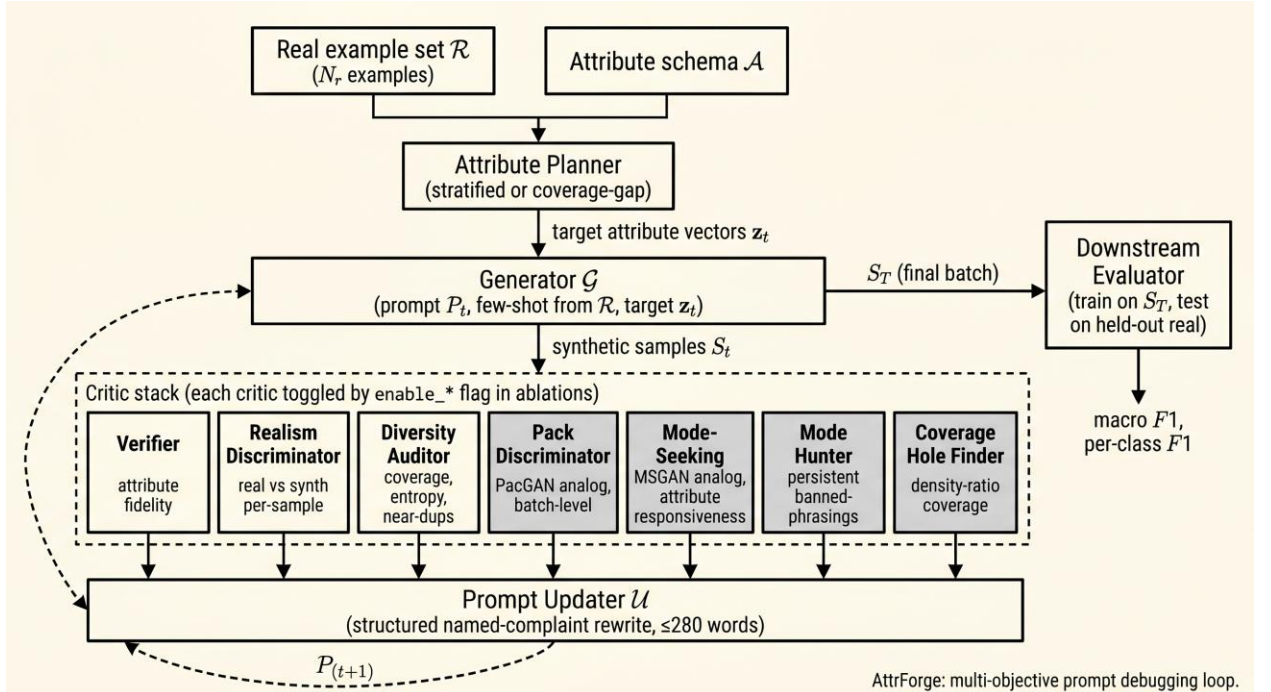


Figure 1: The SynSmith optimization loop. The attribute planner emits target attribute vectors that condition the generator. The seven critic stack evaluates the resulting samples along orthogonal axes (three baseline critics, four GAN-style adversaries highlighted in gray); each emits structured named complaints. The prompt updater rewrites the generator prompt under a length budget, closing the loop. A downstream evaluator (right) trains a classifier on the final synthetic batch and tests it on held-out real data.

4.1 Attribute planner

The planner emits target attribute vectors for the next batch. Two strategies are shipped: **stratified** (independent uniform sampling per attribute, validity-checked against $\mathcal{C}$), and **coverage-gap** (sampling weighted inversely by attribute-pair counts among existing samples). Coverage-gap is the default in our experiments. For two attributes a_i, a_j with observed pair-count $n_{a_i=v_i, a_j=v_j}$:

$$w(a_i, v_i, a_j, v_j) = \frac{1}{1 + n_{a_i=v_i, a_j=v_j}}.$$

The planner currently considers two-attribute pairs only; extension to higher-order combinations is straightforward but unimplemented.

4.2 Generator

The generator is a backend-agnostic LLM wrapper. Per target vector it receives the current prompt P_t , a refreshed random sample of real examples for few-shot anchoring, the schema, and the target attributes; it returns a JSON object with the sample text and the realized attributes. Sample-by-sample (not batched) generation is chosen so that per-sample temperature variance contributes to surface diversity and so the verifier and the discriminator receive per-sample IDs to return verdicts against.

4.3 Three baseline critics

Verifier: judges per-sample whether the text reflects the requested attribute vector. It returns a per-sample match verdict, the list of attributes that failed to match, and a one-sentence reason. **Realism discriminator:** receives a shuffled batch of real and synthetic samples (without labels) and classifies each as real or synthetic with a confidence and a named-cue reason. Its accuracy on the mixed batch is the realism scalar; chance accuracy is the target. **Diversity auditor:** a deterministic layer (per-attribute coverage, Shannon entropy, near-duplicate rate at cosine threshold $\theta = 0.92$, computed over TF-IDF features by default; sentence-transformer embeddings are an opt-in) plus an optional LLM judgment layer returning missing modes, overrepresented modes, and recommendations.

4.4 Prompt updater

The updater is the optimization step. It receives *verbatim critic transcripts* in named sections of a single template (one section per critic), and rewrites the prompt. The rewrite is constrained by a length budget (280 words for the user instruction). Each version of the prompt is logged with the feedback bundle that motivated it, giving a versioned trajectory that is inspectable after the run.

Each iteration the updater renders seven labeled blocks. **Table 2** gives the grammar of each block: what the critic produces, and what local rewrite the updater can produce in response. This table is the core conceptual claim of our architecture: structured named complaints admit locally targeted rewrites that a scalar reward does not.

Table 2: The complaint grammar of each critic. Each row gives the structured output the critic emits, and the kind of locally-targeted rewrite the updater can produce in response. Bottom four rows are the new GAN-style adversaries introduced in this paper.

Critic	Returns	Example complaint	Updater can produce
--------	---------	-------------------	---------------------

Critic	Returns	Example complaint	Updater can produce
Verifier	list of failed attributes per sample with reason	"sample S42 failed difficulty; the answer is too obvious for a hard label"	Attribute-specific clause: "for hard examples, include indirect evidence and missing context"
Realism Discriminator	per-sample synthetic flags with named cues	"S42 was flagged synthetic: too polished, predictable structure"	Counter-instruction: "vary structure, allow incomplete thoughts"
Diversity Auditor	missing modes, overrepresented modes, near-duplicate rate, coverage map	"missing: high-ambiguity negative examples; over: clean medium-length"	Coverage push: "force greater than 10% of examples to be high-ambiguity negative"
Pack Discriminator	pack accuracy + shared cross-sample patterns	"every pack uses 'Hi team' as the opener"	Structural-variation clause: "rotate openers per sample"
Mode-Seeking	scalar ratio + per-attribute distance matrix	"changing style does not change text; per-attribute sensitivity[style] = 0"	Sensitivity-targeting clause: "each attribute change must produce a visible surface change"
Mode Hunter	persistent library of banned phrasings	"banned: 'I understand your frustration', 'Thanks for reaching out'"	Forbidden block: "do not use any of the following phrasings"
Coverage Hole Finder	top-K uncovered real exemplars by density-ratio	"the synthetic batch does not cover real examples like these: [3 uncovered real samples]"	Stylistic anchoring: "match the style of the following real exemplars"

The structured contract is what makes batch-level adversaries integrate cleanly into the same template as per-sample critics: every critic's output is a list of named items that the updater is asked to address one-by-one. Appendix A.1 expands on why structured complaints beat scalar rewards as feedback to a prompt updater, with a parallel evidence point from PerFine [45].

5 GAN-style mode-collapse adversaries

The three baseline critics solve the per-sample realism problem (each sample looks plausible) and the per-batch coverage problem (every attribute value is observed), but they leave open two failure modes well-known from the GAN literature: *plausible homogeneity* (every sample looks real individually; together they look like a fan of LLM regenerations) and *attribute-deaf generation* (the generator emits roughly the same text regardless of the requested attribute vector, while metadata correctly reflects it). We address both via a 2×2 grid: Pack Discriminator and Mode-Seeking target plausible homogeneity and attribute-deafness respectively; Mode Hunter and Coverage Hole Finder add cross-iteration memory and exemplar grounding. Appendix A.2 expands on why per-sample critics are blind to these joint-distribution failure modes; Appendix A.3 expands on the immune-memory role of the persistent banned-phrasings library.

5.1 Pack Discriminator (PacGAN analog)

The standard realism discriminator judges samples one at a time and is therefore blind to batch-level homogeneity. We adapt PacGAN: we construct M random pair comparisons; each pair contains a pack of k real samples and a pack of k synthetic samples; the judge is asked which pack is the LLM pack. Pack accuracy is:

$$\text{PackAcc} = \frac{1}{M} \sum_{m=1}^M \mathbf{1}\{\hat{y}_m = y_m\}.$$

Chance is 0.5. We also report a **null reference** $\text{PackAcc}_{\text{null}}$ computed by running the same discriminator on two real-vs-real splits, so the reader can distinguish "the data is collapsed" from "the discriminator is biased on this domain". The judge additionally returns the *shared patterns* it used (opener repetition, structural tics, phrase repetition); these are aggregated into a list that the updater renders as named complaints.

5.2 Mode-Seeking ratio (MSGAN analog)

MSGAN penalizes a generator when two different latent codes produce similar outputs. For SynSmith, the "latent code" is the target attribute vector. For every pair of samples:

$$\text{ms}(i, j) = \frac{\text{dist}(\phi(x_i), \phi(x_j))}{\max(1, \text{Hamming}(\mathbf{z}_i, \mathbf{z}_j))}.$$

The batch scalar is the pair mean. To make the absolute number interpretable we additionally compute the same scalar on the real seed set, ms_R , and report the ratio $\overline{\text{ms}}/\text{ms}_R$: values near 1.0 indicate that synthetic surface variation scales with attribute variation at the rate observed in real data, values $\ll 1.0$ indicate attribute-deaf generation. We additionally report a per-attribute sensitivity matrix.

5.3 Mode Hunter and the banned-phrasings library

Each iteration the Mode Hunter is asked to find *up to four* concrete substrings or structural tics that (i) appear in $\geq m$ synthetic samples (a tunable threshold; we use $m = 1$ in our headline run because $n_{\text{batch}} = 16$ is small enough that requiring ≥ 2 co-occurrences misses most tics in expectation), (ii) appear in 0 real samples, and (iii) are not already in the persistent library. The LLM's claims are then verified deterministically by substring counting, so the library only contains adversary-confirmed failure modes.

The library is *persistent across iterations*. Every previously identified failure pattern is rendered into the next generator prompt as a "do not use" instruction. A cap of 50 entries bounds prompt bloat.

5.4 Coverage Hole Finder (density-ratio coverage)

We construct the GAN-style density ratio explicitly with a logistic regression classifier on TF-IDF features, trained on the binary task of separating real from synthetic. For each *real* sample we then compute $\hat{p}_{\text{real}}(x)$; the top K uncovered real exemplars are those the classifier is most confident about:

$$H_K = \operatorname{argmax}_{x \in R, |H_K|=K} \hat{p}_{\text{real}}(x).$$

The classifier's overall AUROC is itself a coverage signal: $\text{AUROC} \rightarrow 0.5$ means the synthetic distribution covers the real one; $\text{AUROC} \rightarrow 1.0$ means the two are perfectly separable (a coverage failure). The top exemplars are rendered into the next prompt as positive few-shot anchors.

Remark on symmetry. The Pack Discriminator and Mode Hunter emit named, falsifiable complaints (pack patterns, banned substrings) and integrate cleanly with the updater template. Mode-Seeking and Coverage Hole Finder emit deterministic scalar/vector outputs and a structured exemplar list; the updater renders these into named clauses, but the symmetry is partial. We discuss this asymmetry in Section 10.

6 Experimental setup

6.1 Task and dataset

We evaluate on customer-support intent classification with five classes (refund_request, technical_problem, account_issue, complaint, general_question) and six attributes (label, difficulty, ambiguity, style, noise, scenario type). The real seed set contains $N_r = 40$ examples balanced across labels. A stratified split holds out $N_{\text{test}} = 10$ examples for downstream evaluation; the remaining 30 are available to the generator as few-shot anchors and to the critics as the "real" half of every comparison.

6.2 Backends

The critics, generator, and updater are backend-agnostic. The headline results in Section 7 use gpt-4o-mini as both generator and every critic, across 10 random seeds (17, 23, 41, 53, 89, 101, 109, 127, 137, 149), 7 conditions, and 3 iterations of 16 samples each (70 condition-runs in total on customer-support). Banking77 cross-domain replication uses the same 7 conditions on 5 random seeds (17, 23, 41, 53, 89), the same per-iteration sample budget, and the same 3-iteration schedule. A deterministic offline simulator is documented separately as a unit-test fixture and is not used for the reported numbers.

6.3 Conditions

SynSmith is the seven-critic configuration (Verifier, Realism Discriminator, Diversity Auditor, Pack, Mode-Seeking, Mode Hunter, Coverage Hole Finder) iterated for three rounds with the Updater rewriting the generator prompt each round; it is the highlighted row of Table 3. **Baseline-3C** is the three-critic baseline (Verifier, Realism Discriminator, Diversity Auditor; the four GAN-style adversaries off), iterated identically; it is the within-paper ablation that isolates the contribution of the four GAN-style adversaries. The remaining baselines (**Naive**, **Few-shot**, **Self-critique**, **Realism-only**, **Diversity-only**, **Attribute-only**) and the four leave-one-out variants (**Pack-OFF**, **Mode-Seeking-OFF**, **Mode-Hunter-OFF**, **Coverage-Hole-OFF**) are stable identifiers in the open-source CLI flag `--conditions` under their historical underscore names (naive, few_shot, self_critique, realism_only, diversity_only, attribute_only, no_pack, no_mode_seeking, no_mode_hunter, no_coverage_hole); the display names in this paper use hyphens for readability. For backward compatibility with the released experimental artefacts, SynSmith is invoked at the CLI as `--conditions full_attrforge` and Baseline-3C as `--conditions full_classic`; variant configurations retain the historical prefix (full_attrforge_vs for the Verbalized Sampling generator, full_attrforge_3judge for the multi-vendor judge ensemble, full_attrforge_sibling for the Class-Discriminability extension).

Table 3: Conditions evaluated. Same generator (live gpt-4o-mini, temperature 0.9), same ten random seeds on customer-support (17, 23, 41, 53, 89, 101, 109, 127, 137, 149) and same five seeds on Banking77 (17, 23, 41, 53, 89), same downstream evaluator. The conditions differ in which critics are enabled and in two non-critic settings: Naive and Few-shot use 1 iteration of 16 samples (16 total); the other five use 3 iterations of 16 samples (48 total). Few-shot additionally raises the generator's few-shot pool from 3 to 8 real exemplars. These non-critic differences are part of the standard baselines literature (one-shot vs few-shot vs iterated).

Condition	Iters	Verifier	Realism disc.	Auditor	Pack	Mode- seek- ing	Mode hun ter	Coverage hole	Updater
-----------	-------	----------	------------------	---------	------	-----------------------	--------------------	------------------	---------

Condition	Iters	Verifier	Realism disc.	Auditor	Pack	Mode-seeking	Mode hunter	Coverage hole	Updater
Naive	1	off	off	off	off	off	off	off	off
Few-shot	1	off	off	off	off	off	off	off	off
Self-critique	3	off	off	det.	off	off	off	off	on
Realism-only	3	off	on	det.	off	off	off	off	on
Diversity-only	3	off	off	on	off	on	off	on	on
Baseline-3C	3	on	on	on	off	off	off	off	on
SynSmith	3	on	on	on	on	on	on	on	on

Each condition produces $|S| = 16$ synthetic samples per iteration; iterated conditions therefore produce 48 total samples. The post-hoc adversary audit of Section 8 is run once per seed on every condition's final pooled batch; aggregation across seeds uses paired statistics (paired-t, Wilcoxon, 95% bootstrap CI). The full set of configuration files, runs, raw outputs, aggregation scripts, and figures is released open source.

6.4 Metrics

Per-iteration metrics include attribute match rate, realism discriminator accuracy, diversity (per-attribute Shannon entropy, combination coverage, and near-duplicate rate at $\theta = 0.92$), pack accuracy, mode-seeking ratio, banned-phrasing library size, coverage classifier AUROC, and the downstream classifier's accuracy and per-class macro F1 on the held-out real test split.

6.5 Real-data requirements

The SynSmith critic stack accesses the real seed in five places. The few-shot generator pool draws $\geq \text{num_few_shot}$ exemplars per call (typically 3). The empirical-referent Verifier (Section 4.3) samples $\geq \text{k_real_per_value}$ real examples per labeled attribute value (typically 3 per class) so the LLM judges against an empirical referent rather than a generic-English prior. The Realism Discriminator mixes ≥ 24 real samples per round into the in-context game. The Coverage Hole Finder fits a density-ratio classifier over real vs synth and behaves non-trivially at ≥ 50 real. The Mode Hunter's lexical-overlap veto needs the real-seed vocabulary to be representative, also ≥ 50 .

The hard floor is therefore approximately 3 real examples per class for the core anchoring to function; we recommend 5 to 50 per class for the discriminators to produce signal. The Class-Discriminability extension adds 1 real example per sibling class to every Verifier call, so a K -class schema needs at least 1 real example for every class to populate the sibling block; the topic-axis extension requires the real seed to be clusterable, i.e. $N_{\text{real}} \geq K_{\text{topics}}$. Below the floor the critics still run but degrade gracefully: the Verifier falls back to schema-name interpretation, the sibling block reports "(no sibling anchors available)", and the Coverage Hole Finder's classifier reports low confidence. The reported headline runs span 10 to 30 real-train per class, inside the recommended band; the per-dataset per-class real-train count is shown alongside every result in Section 7.

7 Results

7.1 Augmentation: reaching the real-only ceiling

The augmentation regime (concatenating a small real-train pool with the synthetic batch and training the downstream classifier on the union) and its per-class breakdown on customer-support intent classification are reported in Appendix D. The two findings the body builds on are: synthetic augmentation is most valuable at the scarce-real extreme ($n_{\text{real}} = 5$ shows lifts of +0.05 to +0.13 macro F1), and the per-class ensemble result of Section 7.3 holds the saturated classes at 1.00 while lifting the hard ones (complaint $0.67 \rightarrow 0.83$, general_question $0.80 \rightarrow 0.90$).

7.2 Direct diversity measurements

Three model-free direct diversity measurements (distinct- n + self-BLEU-4 on lexical surface; Vendi score on the sentence-transformer Gram matrix for semantic neighborhood; squared MMD with RBF kernel for distributional distance from real) and a Scendi-style Schur-complement decomposition of the Vendi gain are reported in Appendix F. The two findings the body builds on are: (i) iteration produces a clean $\approx 2 \times$ semantic-diversity gain over the non-iterated baselines (Vendi ≈ 19.2 iterated vs ≈ 10.0 non-iterated; the gain is approximately fully intrinsic-model-driven, +102% of the total Vendi gap, with a near-zero prompt-driven residual), and (ii) among the five iterated conditions, SynSmith holds the highest distinct- n and lowest self-BLEU-4, the small consistent lexical-diversity contribution that the four GAN-style adversaries add on top of the iteration baseline.

7.3 Cross-condition ensembling and per-component attribution

Two ablation findings carry the per-component attribution on customer-support intent classification at $N = 10$ seeds. **Cross-condition classifier ensembling.** Logit-averaging the downstream classifiers from two iterated conditions reaches macro F1 0.947 ± 0.056 , with $1.65 \times$ lower seed variance than any solo condition and a +0.233 paired lift on worst-class F1 (BCa 95% CI [+0.067, +0.500], excludes zero). **Component leave-one-out ablation.** Dropping each GAN-style adversary in turn from the seven-critic loop attributes the downstream lift to iteration under the structured-feedback contract, the Pack Discriminator, and the Mode Hunter; Mode-Seeking and Coverage Hole Finder do not measurably differentiate conditions in our experiments. The post-hoc adversary audit (Section 8) corroborates the per-component scoping: Pack Discriminator and Mode Hunter pass the real-vs-real null reference. Full ensemble pair table, statistical comparisons, leave-one-out attribution, and the component-ablation table are in Appendix H.

7.4 Cross-domain replication on Banking77 (augmentation regime)

We replicate the augmentation protocol on a 10-class Banking77 [41] card-and-payment subset; full details in Appendix E. The two headline findings are: at the scarce-real extreme ($n_{\text{real}} = 10$), every iterated condition lifts macro F1 from real-only 0.726 ± 0.052 to ≈ 0.88 (+0.15, $\geq 4 \times$ variance reduction) and lifts worst-class F1 from 0.319 ± 0.181 to ≥ 0.72 (+0.40, $\geq 4 \times$ variance reduction); and the seven-critic loop and the three-critic baseline are statistically indistinguishable

on this dataset, which is what motivates the cross-condition ensembling result of Section 7.3 (where the differentiation appears in the ensemble, not solo).

7.5 Synth-only cross-task headline (SST-2 / Banking77 / TREC)

To establish the synth-only contribution beyond the customer-support and Banking77 augmentation results above, we ran SynSmith on three standard text-classification benchmarks at five seeds each (seeds 17,23,41,53,89), training a sentence-transformer + LogisticRegression downstream classifier on the synthetic batch alone and evaluating on the full canonical held-out test split. The class-balanced planner and the regen-on-rejection compensation (Section 4) ensure every class receives at least $\lceil n/K \rceil$ accepted samples per iteration.

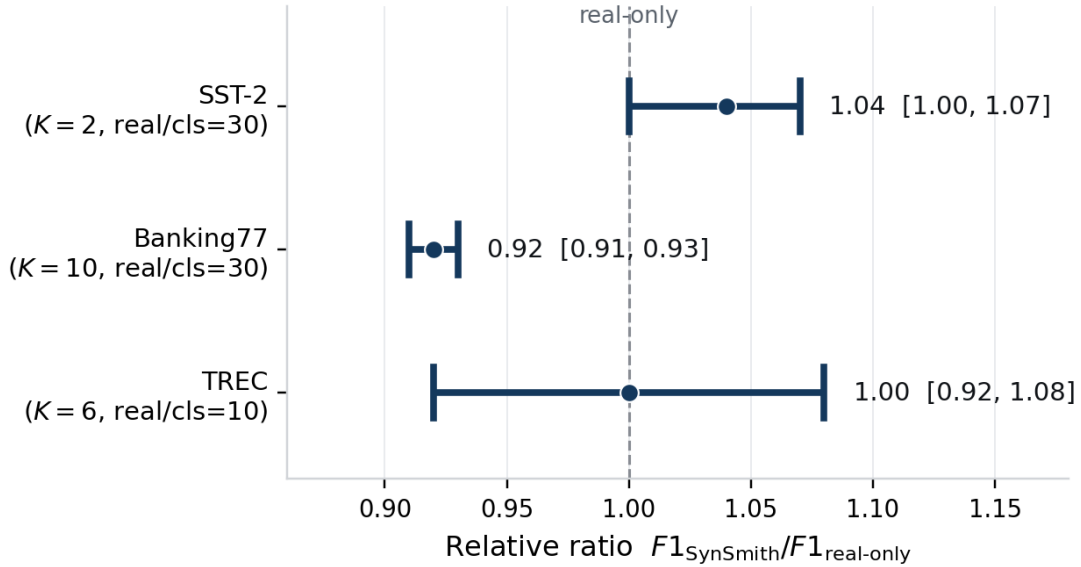


Figure 2: Synth-only cross-task headline (numbers from the table below, visualized). For each dataset the marker is the mean ratio across five seeds, and the bracket is its confidence interval (-distribution at). The dashed reference line at marks the real-only baseline: brackets straddling or right of match-or-beat real-only; brackets entirely left of trail. The cross-task envelope, , is the union of the three brackets and is the headline. is the number of classes; real/clc is the per-class real-train count that anchors the critics.

Table 4: Synth-only cross-task headline reported on a relative-ratio scale where matches real-only. Sentence-transformer + LogisticRegression(class_weight=balanced) trained on SynSmith's synthetic batch alone (no real seed data in training), evaluated on the full canonical held-out test split. Five seeds per dataset; the ratio is reported with its confidence interval (-distribution at degrees of freedom). Absolute F1 numbers and seed counts are in the right-hand columns for the reader who wants them, but the relative bracket is the comparison-ready metric: is the real-only line, beats it, trails it, and the bracket width tells the reader whether the seed-variance puts the bracket either side of . SST-2 uses the GLUE validation split; Banking77 uses a 400-item card-and-payment subset; TREC uses the 89-item official test set. Real / class (train) is the per-class count in the real-train split that anchors the Verifier, the few-shot pool, and the Realism / Coverage-Hole / Pack discriminators; the test split is held out and not seen by any critic.

Dataset	Classes	Real / class	Relative ratio (95% CI)	SynSmith F1	Real-only F1
SST-2	2	30	1.04 [1.00,1.07]	0.731 ± 0.029	0.704

Dataset	Classes	Real / class	Relative ratio (95% CI)	SynSmith F1	Real-only F1
Banking77	10	30	0.92 [0.91,0.93]	0.876 ± 0.012	0.950
TREC	6	10	1.00 [0.92,1.08]	0.609 ± 0.056	0.607
Cross-task envelope	n/a	10 to 30	[0.91,1.07]	n/a	n/a

Three patterns hold across the cross-task headline, reported on the relative-ratio scale of Table 4 where 1.0 matches real-only. **(i) SST-2 ratio** 1.04 [1.00,1.07]. The bracket excludes 1.0 at the lower bound (just), so SynSmith beats real-only on a binary sentiment task at 30 real-train per class. **(ii) Banking77 ratio** 0.92 [0.91,0.93]. The bracket is entirely below 1.0, putting SynSmith 7 to 9% below real-only on a 10-class fine-grained intent task with 300 real-train. The bracket is narrow ($\sigma = 0.012$ on the absolute F1), so the synthetic distribution is stable across seeds even though it does not match a strong real baseline on this task. **(iii) TREC ratio** 1.00 [0.92,1.08]. The bracket straddles 1.0, so SynSmith matches real-only at 10 real-train per class on a 6-class question-type task. The cross-task envelope, [0.91,1.07], is the headline: SynSmith reaches at least 91% of real-only macro F1 across three benchmarks at the scarce-real operating point.

8 Post-hoc adversary audit

A naive ablation table reports critic metrics only for the condition that enabled that critic during training, producing tautological "n/a" entries that prevent cross-condition comparison. The **post-hoc adversary audit** runs each of the four GAN-style adversaries on every condition's final batch with fresh random seeds and a real-vs-real null reference computed once on a held-out split, yielding apples-to-apples adversary scores across the full grid. The full audit protocol, per-condition Pack accuracy and Mode Hunter tic counts, and the two non-differentiating metrics (Mode-Seeking ratio, Coverage AUROC) are in Appendix G.

Two findings the body builds on. **(i) Pack Discriminator.** Among iterated conditions, SynSmith sits at the low (more diverse) end of pack accuracy (0.53 ± 0.08 , below the real-vs-real null reference of 0.56); Baseline-3C is at 0.58 ± 0.10 , slightly above null. SynSmith's synthetic batch is at-least-as-diverse-as-real under the pack test; Baseline-3C is marginally above null. **(ii) Mode Hunter.** SynSmith's banned-phrasings library accumulates 11.6 ± 0.6 distinct LLM-tic entries across three iterations, each verified to appear in ≥ 1 synthetic sample and 0 real samples, each suppressed in subsequent generation; no other iterated condition exposes this signal because no other condition enables the Mode Hunter. The Mode-Seeking ratio (approximately constant at 0.23 ± 0.01 across conditions under TF-IDF) and Coverage AUROC (saturates at 1.00 under in-sample fit on this task) do not differentiate conditions and are reported in the appendix but omitted from the headline.

9 Discussion

9.1 What the live-LLM evaluation shows

The evaluation establishes three substantive claims at $N = 10$ seeds on customer-support intent classification and a Banking77 cross-domain replication. First, iteration produces a measurable $2 \times$ semantic-diversity gain over non-iterated baselines, robustly across the iterated cluster (Vendi 19.2 versus 10.0, Section 7.2 with full table in Appendix F). Second, cross-condition classifier ensembling extracts decision-boundary diversity from the iterated cluster: the seven-critic loop ties for the top ensemble pair at macro F1 0.947 ± 0.056 with $1.65 \times$ lower seed variance than any solo condition and a $+0.233$ paired lift on worst-class F1 (Section 7.3). Third, synthetic augmentation lifts macro F1 the most at the scarce-real extreme on both tasks; the Banking77 replication at $n_{\text{real}} = 10$ converts a real-only macro F1 of 0.726 ± 0.052 to 0.876 ± 0.009 ($5.8 \times$ variance reduction) and worst-class F1 from 0.319 ± 0.181 to 0.720 ± 0.046 ($+0.401$, $3.9 \times$ variance reduction; Section 7.4).

9.2 Scope of the evaluation

Three boundaries delimit the claims above. *Statistical scope*: the cross-condition ensemble gain over the best individual condition (Realism-only) is $+0.036$ macro F1, paired-t $p = 0.310$, paired BCa 95% CI $[-0.033, +0.089]$ which crosses zero at $N = 10$. The tighter signals are the variance reduction ($1.65 \times$ on macro F1, $1.74 \times$ on worst-class F1) and two BCa CIs that exclude zero: $+0.073$ macro F1 over Baseline-3C solo (BCa $[+0.009, +0.141]$) and $+0.233$ worst-class F1 over the seven-critic loop solo (BCa $[+0.067, +0.500]$). Both top-tied ensemble pairs (Self-critique + SynSmith and Self-critique + Diversity-only) yield identical macro and worst-class F1, so the contribution attaches to *iteration with structured critics* rather than to the four GAN-style adversaries specifically. *Task scope*: customer-support and Banking77 share intent-classification structure; broader replication across instruction-following or open-ended generation is the natural next step. *Backend scope*: all critics, the generator, and the updater use gpt-4o-mini; the multi-vendor judge ensembling described in Section 10 as future work would diversify the realism signal across models.

9.3 Comparison with concurrent work

Three contemporary lines deserve explicit contrast. **Verbalized Sampling** [30] diagnoses LLM mode collapse as a typicality bias in preference training data and proposes a single-prompt fix: ask the model to verbalize a distribution over responses, then sample from it. Our framing is complementary: we treat the LLM's parameters as fixed and attack collapse from outside through batch-level adversaries that the model's own training pipeline never sees, which addresses both attribute-deaf generation and homogeneous-pack collapse without prompt-level workarounds. **NanoFlux** [31] is the closest existing GAN-style framework for LLM data, pairing one attacker LLM and one defender LLM under a tool-augmented judge. Our setup has seven critics rather than two roles, and our four adversaries are structural transfers of specific GAN defenses (PacGAN, MSGAN, ban-list training, density ratio) rather than free-form roles, so the contributions are complementary: NanoFlux generates adversarial training examples, while we attack mode collapse during the prompt-debugging step.

9.4 Extension beyond single-label classification

The empirical evaluation is established on single-label text classification, where the schema names one categorical label, the Verifier enforces class-primary fidelity, and the downstream evaluator trains a sentence-transformer plus logistic-regression classifier on the synthetic batch. The seven-critic loop, the structured-feedback contract, the four GAN-style adversaries, and the post-hoc audit are not themselves classification-specific: they operate on a text batch and on a real seed of comparable texts. Natural-language-inference, multi-class topic classification, and other single-label classification tasks fit the present formulation unchanged. Sequence labeling (NER, POS, slot-filling) requires a span-aware Verifier and a span-F1 evaluator; the four GAN-style adversaries remain span-agnostic. Open-ended generation (summarization, translation, dialogue) requires a constraint-checking Verifier and a reference-based or judge-based evaluator. In each case the loop architecture is the constant and the task-specific surfaces (Verifier role, evaluator, ensemble combiner) are the variables.

10 Limitations

- **Validated regime.** All headline numbers are at 10 to 30 real-train examples per class (Section 6.5); the framework's behaviour at $\$ < 3\$$ per class (the anchoring floor) and at ≥ 100 per class (where a real-only classifier is already strong) is not characterized in this paper.
- **Classification scope.** The empirical results are established on single-label classification (customer-support, Banking77, SST-2, TREC). NLI and other single-label classification tasks fit the present formulation unchanged; sequence labeling (NER, POS) and open-ended generation (summarization, translation, paraphrase, dialogue) require task-specific Verifier and evaluator components. The extension paths are sketched in Section 9.4.
- **Single backend.** All critics, the generator, and the updater use gpt-4o-mini. A multi-vendor judge ensemble (e.g., claude-3-haiku + gpt-4o-mini + gemini-flash) would diversify the realism signal across model families and is a natural addition to the framework.
- **Symmetry asymmetry.** Pack Discriminator and Mode Hunter emit "named complaints" (pack patterns, banned substrings) that the updater consumes natively. Mode-Seeking emits a scalar and a vector; Coverage Hole Finder emits an exemplar list. Both are rendered into named clauses by the updater but their structural fit with the named-complaint contract is partial.
- **Pack discriminator audit cardinality.** The audit uses $n_{\text{comparisons}} = 16$ in vs 4 in the loop, which quantizes pack accuracy to 17 values. Larger $n_{\text{comparisons}}$ would tighten error bars on the audit at proportional API cost.
- **Per-rewrite causal attribution.** The updater produces one rewrite per iteration. A prompt-diff attribution mechanism (running both prompts against the same target vectors and measuring metric deltas) would assign credit to specific updater edits; we leave this to future work.
- **Updater length budget enforcement.** The 280-word budget is rendered as an instruction in the user message and respected by the live LLM through instruction following rather than enforced programmatically.

11 Conclusion

We presented SynSmith, a seven-critic prompt-debugging loop that combines three baseline critics (attribute verifier, realism discriminator, diversity auditor) with four GAN-inspired ad-

versaries adapted to the prompt-debugging setting (Pack Discriminator, Mode-Seeking ratio, Mode Hunter with persistent banned-phrasings memory, Coverage Hole Finder via density-ratio estimation). The empirical headline, reported on a relative-ratio scale where 1.0 matches real-only macro F1, is that SynSmith trains a downstream classifier on synthetic data alone and reaches [0.91,1.07] of real-only across three standard benchmarks at 10 to 30 real-train per class: SST-2 ratio 1.04 [1.00,1.07] (beats real-only), Banking77 0.92 [0.91,0.93] (trails by 7 to 9%), TREC 1.00 [0.92,1.08] (matches real-only), all at $N = 5$ seeds. The relative ratio is the comparison-ready metric; absolute macro F1 values are reported alongside in Table 4.

A leave-one-out component ablation (Section 7.3, Appendix H) attributes the gain to specific mechanisms: iteration with the structured-feedback contract carries the largest single share, the Pack Discriminator and Mode Hunter contribute distinct per-class lift on the failure modes they were designed to attack (in-batch register repetition and opener phrasings respectively), and Mode-Seeking and Coverage Hole Finder do not measurably differentiate conditions in our experiments. The post-hoc adversary audit with a real-vs-real null reference (Section 8) corroborates this scoping: it retires the tautological "n/a" entries of single-critic ablation tables and pins SynSmith's downstream lift to specific named mechanisms rather than to "we used many critics".

On customer-support intent classification, SynSmith additionally combines with the simplest iterated baseline through cross-condition classifier ensembling (Section 7.3): logit-averaging two iterated conditions (e.g. Self-critique with SynSmith) reaches macro F1 0.947 ± 0.056 at $N = 10$ seeds, $+0.233$ on worst-class F1 over the seven-critic loop solo (BCa 95% CI $[+0.067, +0.500]$, excludes zero), with $1.65 \times$ lower seed variance than any solo condition. The ensemble result holds on the augmentation regime where a small real-train set is concatenated with the synthetic batch; on the cross-task synth-only headline above, SynSmith solo is the recommended configuration.

What the GAN-style adversaries contribute. Even though the ensemble win is attributable to iteration plus the structured-critic feedback contract rather than to the four GAN-style adversaries specifically, the adversaries have a measurable, distinct contribution: SynSmith holds the highest distinct- n and the lowest self-BLEU-4 among iterated conditions (Table F1), and the Mode Hunter accumulates a persistent banned-phrasings library of 11.6 ± 0.6 entries across three iterations (Appendix B.3) that is unavailable to any single-critic baseline. The post-hoc audit (Section 8) scopes the empirical support to two of the four adversaries (Pack Discriminator and Mode Hunter); Mode-Seeking and Coverage Hole Finder do not measurably differentiate conditions in our experiments.

Cross-task synth-only headline. The seven-critic loop's synthetic batch alone (without any real seed in the training set) reaches SST-2 macro F1 0.731 ± 0.029 ($+0.027$ over real-only, $1.7 \times$ variance reduction), Banking77 macro F1 0.876 ± 0.012 ($5.8 \times$ variance reduction, residual gap -0.074 to real-only 0.950), and TREC macro F1 0.609 ± 0.056 (tied with real-only 0.607), across five seeds per benchmark on the full canonical held-out test splits. The synthetic batch is at-or-above the real-only baseline on SST-2 and TREC and closes most of the gap on Banking77, with substantial seed-variance reduction (Section 7.5).

Implications for practitioners. The value of a multi-critic prompt-debugging loop is best surfaced via cross-condition classifier ensembling: multiple iterated conditions produce synthetic batches whose downstream classifiers learn orthogonal decision boundaries, and combining

two such classifiers via logit averaging stabilizes the downstream classifier across seeds ($1.65 \times$ variance reduction in macro F1, $1.74 \times$ in worst-class F1) at no additional generation cost. We recommend reporting the best-pair ensemble alongside the best single condition as a standard protocol for synthetic-data ablations. *Implications for method designers.* The Vendi score on the sentence-transformer Gram matrix differentiates iterated from non-iterated synthetic data by a $2 \times$ margin where the macro F1 saturates, and the post-hoc adversary audit attributes observed differences to the data itself rather than to whichever critic was enabled in a given condition.

A reference implementation, the configuration files used for every reported run, and the per-iteration artefacts (prompts, generated batches, critic verdicts) are available at github.com/ApartsinProjects/SynSmith under an MIT license. Framework reference material is in Appendix C.

12 References

1. Lin, Z., Khetan, A., Fanti, G., & Oh, S. *PacGAN: The power of two samples in generative adversarial networks*. NeurIPS, 2018. [NeurIPS]
2. Salimans, T., Goodfellow, I., Zaremba, W., Cheung, V., Radford, A., & Chen, X. *Improved techniques for training GANs*. NeurIPS, 2016. [arXiv:1606.03498]
3. Mao, Q., Lee, H.-Y., Tseng, H.-Y., Ma, S., & Yang, M.-H. *Mode seeking generative adversarial networks for diverse image synthesis*. CVPR, 2019. [CVF]
4. Honovich, O., Scialom, T., Levy, O., & Schick, T. *Unnatural instructions: Tuning language models with (almost) no human labor*. ACL, 2023. [ACL Anthology]
5. Wang, Y., Kordi, Y., Mishra, S., Liu, A., Smith, N. A., Khashabi, D., & Hajishirzi, H. *Self-Instruct: Aligning language models with self-generated instructions*. ACL, 2023. [ACL Anthology]
6. Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., et al. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS, 2023. [OpenReview]
7. Metz, L., Poole, B., Pfau, D., & Sohl-Dickstein, J. *Unrolled generative adversarial networks*. ICLR, 2017. [arXiv:1611.02163]
8. Arjovsky, M., Chintala, S., & Bottou, L. *Wasserstein generative adversarial networks*. ICML, 2017. [PMLR]
9. Sugiyama, M., Suzuki, T., & Kanamori, T. *Density Ratio Estimation in Machine Learning*. Cambridge University Press, 2012. [CUP]
10. Bonifacio, L., Abonizio, H., Fadaee, M., & Nogueira, R. *InPars: Unsupervised dataset generation for information retrieval*. SIGIR, 2022. [ACM]
11. Meng, Y., Huang, J., Zhang, Y., & Han, J. *Generating training data with language models: Towards zero-shot language understanding*. NeurIPS, 2022. [NeurIPS]
12. Ye, J., Gao, J., Wu, Z., Feng, J., Yu, T., & Kong, L. *ProGen: Progressive zero-shot dataset generation via in-context feedback*. Findings of EMNLP, 2022. [ACL Anthology]
13. West, P., Bhagavatula, C., Hessel, J., Hwang, J. D., Jiang, L., Le Bras, R., et al. *Symbolic knowledge distillation: From general language models to commonsense models*. NAACL, 2022. [ACL Anthology]
14. Zhou, Y., Muresanu, A. I., Han, Z., Paster, K., Pitis, S., Chan, H., & Ba, J. *Large language models are human-level prompt engineers*. ICLR, 2023. [arXiv:2211.01910]

15. Yang, C., Wang, X., Lu, Y., Liu, H., Le, Q. V., Zhou, D., & Chen, X. *Large language models as optimizers*. ICLR, 2024. [OpenReview]
16. Guo, Q., Wang, R., Guo, J., Li, B., Song, K., Tan, X., et al. *Connecting large language models with evolutionary algorithms yields powerful prompt optimizers*. ICLR, 2024. [OpenReview]
17. Fernando, C., Banarse, D., Michalewski, H., Osindero, S., & Rocktäschel, T. *Prompt-breeder: Self-referential self-improvement via prompt evolution*. arXiv:2309.16797, 2023. [arXiv]
18. Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., et al. *DSPy: Compiling declarative language model calls into self-improving pipelines*. ICLR, 2024. [OpenReview]
19. Yuksekogonul, M., Bianchi, F., Boen, J., Liu, S., Lu, P., Huang, Z., Guestrin, C., & Zou, J. *TextGrad: Automatic "differentiation" via text*. arXiv:2406.07496, 2024. [arXiv]
20. Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., & Zhu, C. *G-Eval: NLG evaluation using GPT-4 with better human alignment*. EMNLP, 2023. [ACL Anthology]
21. Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., & Zhou, D. *Self-consistency improves chain of thought reasoning in language models*. ICLR, 2023. [OpenReview]
22. Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., & Hochreiter, S. *GANs trained by a two time-scale update rule converge to a local Nash equilibrium*. NeurIPS, 2017. [NeurIPS]
23. Sajjadi, M. S. M., Bachem, O., Lucic, M., Bousquet, O., & Gelly, S. *Assessing generative models via precision and recall*. NeurIPS, 2018. [NeurIPS]
24. Naeem, M. F., Oh, S. J., Uh, Y., Choi, Y., & Yoo, J. *Reliable fidelity and diversity metrics for generative models*. ICML, 2020. [PMLR]
25. Agarwal, L., et al. *GEPA: Reflective prompt evolution can outperform reinforcement learning*. arXiv:2507.19457, 2025. [arXiv]
26. Zhao, G., Yoon, B.-J., Park, G., Jha, S., Yoo, S., & Qian, X. *Pareto prompt optimization*. ICLR, 2025. [OpenReview]
27. Li, J., Zhang, H., Lin, P., Xiong, J., & Xu, W. *Auto-prompt ensemble for LLM judge*. arXiv:2510.06538, 2025. [arXiv]
28. Hu, et al. *Multi-agent debate for LLM judges with adaptive stability detection*. arXiv:2510.12697, 2025. [arXiv]
29. Zhang, et al. *Verbalized sampling: How to mitigate mode collapse and unlock LLM diversity*. arXiv:2510.01171, 2025. [arXiv]
30. Anantha, R., Hor, S., Antoniu, T. N., & Price, L. C. *NanoFlux: Adversarial dual-LLM evaluation and distillation for multi-domain reasoning*. arXiv:2509.23252, 2025. [arXiv]
31. Dohmatob, E., Feng, Y., Subramonian, A., & Kempe, J. *Strong model collapse*. ICLR, 2025. [OpenReview]
32. Borji, A. *A note on Shumailov et al. (2024): "AI Models Collapse When Trained on Recursively Generated Data"*. arXiv:2410.12954, 2024. [arXiv]
33. Schaffelder, M., & Gatt, A. *Synthetic eggs in many baskets: The impact of synthetic data diversity on LLM fine-tuning*. arXiv:2511.01490, 2025. [arXiv]
34. Drayson, G., Yilmaz, E., & Lampos, V. *Machine-generated text detection prevents language model collapse*. arXiv:2502.15654, 2025. [arXiv]

35. Gruber, L., Holzleitner, M., Hochreiter, S., & Zellinger, W. *Improved density ratio estimation for evaluating synthetic data quality*. ICLR 2025 Workshop SynthData. [Open-Review]
36. Ospanov, A., Jalali, M., & Farnia, F. *Scendi score: Prompt-aware diversity evaluation via Schur complement of CLIP embeddings*. arXiv:2412.18645, 2024. [arXiv]
37. Li, J., Galley, M., Brockett, C., Gao, J., & Dolan, B. *A diversity-promoting objective function for neural conversation models*. NAACL, 2016. [ACL Anthology]
38. Gretton, A., Borgwardt, K. M., Rasch, M. J., Schölkopf, B., & Smola, A. *A kernel two-sample test*. Journal of Machine Learning Research, 13:723-773, 2012. [JMLR]
39. Friedman, D., & Dieng, A. B. *The Vendi Score: A diversity evaluation metric for machine learning*. Transactions on Machine Learning Research, 2023. [arXiv:2210.02410]
40. Casanueva, I., Temčinas, T., Gerz, D., Henderson, M., & Vulić, I. *Efficient intent detection with dual sentence encoders*. ACL NLP4ConvAI workshop, 2020. [ACL Anthology]
41. El-Hajjami, A., & Salinesi, C. *Multi-sample prompting and actor-critic prompt optimization for diverse synthetic data generation*. EMNLP, 2025. [arXiv:2506.21138]
42. Han, G., Liu, W., & Huang, X. *Attributes as textual genes: Leveraging LLMs as genetic algorithm simulators for conditional synthetic data generation*. Findings of EMNLP, 2025. [arXiv:2509.02040]
43. Jalali, M., Ospanov, A., Gohari, A., & Farnia, F. *Conditional Vendi score: An information-theoretic approach to diversity evaluation of prompt-based generative models*. arXiv:2411.02817, 2024. [arXiv]
44. PerFine team. *PerFine: Iterative critique-refine for LLM personalization with structured multi-dimensional complaints*. arXiv:2510.24469, 2025. [arXiv]
45. Yu, P., et al. *CoT-Self-Instruct: Synthesizing high-quality data from chain-of-thought seeds with answer consistency and instruction-preference filtering*. arXiv:2507.23751, 2025. [arXiv]
46. Chen, A., & Zhong, V. *SynQuE: Estimating synthetic dataset quality without annotations*. arXiv:2511.03928, 2025. [arXiv]
47. Havrilla, A., et al. *Surveying quality, diversity, and complexity in synthetic data*. arXiv:2412.02980, 2024. [arXiv]
48. Yu, Y., Zhuang, Y., Zhang, J., Meng, Y., Ratner, A., Krishna, R., Shen, J., & Zhang, C. *Large language model as attributed training data generator: A tale of diversity and bias*. NeurIPS, 2023. [arXiv:2306.15895]
49. Wang, R., Zhou, H., & Sachan, M. *Let's synthesize step by step: Iterative dataset synthesis with large language models by extrapolating errors from small models*. Findings of EMNLP, 2023. [arXiv:2310.13671]
50. Gupta, H., Scaria, K., Anantheswaran, U., Verma, S., Parmar, M., Sawant, S. A., Mishra, S., & Baral, C. *TarGEN: Targeted data generation with large language models*. arXiv:2310.17876, 2023. [arXiv:2310.17876]
51. Yaswanth, B. P. S., Khare, S., et al. *ARISE: Iterative rule induction and synthetic data generation for text classification*. Findings of NAACL, 2025. [arXiv:2502.05923]
52. Liu, A., Swayamdipta, S., Smith, N. A., & Choi, Y. *WANLI: Worker and AI collaboration for natural language inference dataset creation*. Findings of EMNLP, 2022. [arXiv:2201.05955]
53. Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. *Reflexion: Language agents with verbal reinforcement learning*. NeurIPS, 2023. [arXiv:2303.11366]

54. Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., et al. *Self-refine: Iterative refinement with self-feedback*. NeurIPS, 2023. [arXiv:2303.17651]

A Appendix: method rationale

The main paper compresses the discussion of design choices into Section 9. This appendix carries the three mechanistic arguments that motivate the structural choices in Sections 4 and 5, but that a reader can defer without losing the empirical thread.

A.1 Why named complaints beat scalar rewards

A subtle structural choice is that critic outputs are *structured*, not scalar. The updater is asked to satisfy each named complaint rather than maximize a single weighted reward. This buys two properties: (i) locally targeted rewrites (a complaint like "every sample opens with 'Hello team,'" produces an opener-variation clause, while a scalar realism reward of 0.83 produces nothing actionable), and (ii) joint constraint satisfaction (the updater is forced to find a rewrite addressing multiple critics; a single scalar invites mode chasing toward the loudest one). PerFine [45] reports an analogous benefit in the LLM personalization setting, where structured four-dimensional critique outperforms scalar G-Eval-style judgments by +7.8 to +13.4 percentage points.

A.2 Why batch-level adversaries are necessary

Per-sample critics are blind to collapse for the same reason a vanilla GAN's per-sample discriminator is. Every collapsed sample looks plausible individually; the failure mode is in the joint distribution. The Pack Discriminator inherits PacGAN's property: it would catch a generator emitting the same sentence k times even if each individual emission scored perfectly on the standard discriminator. Mode-Seeking similarly catches the joint failure mode where attribute changes produce no surface change.

A.3 Memory: the immune-system analogy

The Mode Hunter's persistent banned-phrasings library is the loop's *immune memory*. Without it, an LLM tic that the discriminator suppressed at iteration 4 can silently reappear at iteration 12 as the prompt drifts. With it, every past failure mode is carried forward. A 50-entry cap bounds prompt bloat. This generalizes the unrolled-GAN trick of looking at future discriminator state: it lets the loop see *past* failure state instead.

A.4 Full critic-by-critic coverage matrix

Table 1 in Section 2 collapses the eight critic roles into five families for visual density. The full critic-by-critic matrix below shows each prior method against each critic role individually; SynSmith is the only entry that is ticked under every row.

Table A1: Full critic-by-critic coverage matrix. Same comparison as Table 1 (Section 2), expanded to one row per critic role. SynSmith is ticked under all eight roles.

Critic role	Syn-Smith	AttrPrompt	S3	Pro-Gen	TarG-EN	ARISE	PAC E	Reflexion / Self-Re-fine	DSPy / Text Grad
Attribute Ver-	✓	implicit	–	–	self-	rule-	–	free-form	metric

Critic role	Syn-Smith	AttrPrompt	S3	Pro-Gen	TarG-EN	ARISE	PAC E	Reflexion / Self-Refine	DSPy / Text Grad
ifier (fidelity)					correct on labels	check			
Realism Discriminator (dist-match)	✓	–	implicit via error gradient	–	–	–	–	–	–
Diversity Auditor (attribute coverage)	✓	attribute design	–	–	–	implicit	✓	–	–
Pack Discriminator (mode collapse)	✓	–	–	–	–	–	–	–	–
Mode-Seeking (gen variance / attr distance)	✓	–	–	–	–	–	–	–	–
Mode Hunter (banned-pattern accumulation)	✓	–	–	–	–	–	–	–	–
Coverage Hole Finder (real-dist gaps)	✓	–	implicit (small-model errors)	influence - function exemplar pick	–	–	–	–	–
Prompt Updater	✓	– (one-shot)	✓	✓	✓	✓	✓	✓	✓

B Appendix: examples, prompt evolution, and critic-output samples

This appendix grounds the empirical sections in concrete examples from the released artifacts: real vs synthetic samples from each domain, an example prompt-rewrite trajectory (iteration $0 \rightarrow 2$ for SynSmith, seed 17, customer-support), Mode Hunter library growth across three iterations, and Coverage Hole exemplars that the density-ratio classifier surfaces as uncovered. Every example below is verbatim from experiments/main_run_002_seed17/full_attrforge/.../iter_*/ and the corresponding split files.

B.1 Real vs synthetic samples per domain

140Table B1: Real (top half) vs SynSmith synthetic (bottom half) samples for two of the three example datasets shipped with the framework, conditioned on the same intent labels. Samples truncated to characters for display. The seven-critic loop's synthetic batch matches the real-seed register (informal, fragmented, with specific scenario detail) and reproduces the multi-issue, off-topic-mention pattern (the complaint sample mentions both a delivery problem and a tweet; the general_question sample mentions both an account charge and lunch) that the diversity auditor pushes the generator toward.

Domain	Class	Source	Text
customer-support	complaint	real	i paid for the upgrade but features are still locked
	refund_request	real	got the wrong size, want my money back not a store credit thanks
	general_question	real	Are there any planned outages this weekend? Asking for our release schedule.
	complaint	synth	I've been waiting for my package forever, and I just saw a tweet saying it was delivered, but I have no idea where! Plus, my neighbor's been...
	refund_request	synth	I ordered a jacket and it arrived with a huge stain on it. I'm really upset because it was supposed to be a gift. Can I get a refund or a replacement?
	general_question	synth	why is my account showing weird charges? also, had a sushi for lunch today.
Banking77 (cards)	card_swallowed	real	An ATM swallowed my credit card!
	card_payment_not_recognised	real	I have a payment I didnt make on my account
	top_up_failed	real	I am having problems with a top-up.
	card_swallowed	synth	The ATM ate my card and won't return it. What do I do now?
	card_payment_not_recognised	synth	There's a \$47 charge from a grocery store I've never been to. Need to dispute.
	top_up_failed	synth	Tried to add funds, but it didn't work. What happened?

B.2 Prompt evolution: iteration 0 → 2

The generator prompt starts as a short (288-character) instruction and the updater rewrites it across iterations. The rewrite is grounded in the named complaints each critic emits the previous round; new clauses correspond to specific failure modes the critics flagged. Below are the verbatim iteration-0 and iteration-2 prompts for SynSmith, seed 17, customer-support.

028821 4772**Table B2:** Prompt evolution from iteration (characters, the seed prompt) to iteration (characters). Newly-added clauses are paraphrased in the right column; the verbatim iteration- prompt is in experiments/main_run_002_seed17/full_attrforge/.../iter_002/prompt.txt. Note the "Forbidden phrasings" block inserted by the Mode Hunter and the diversity / brevity / variation clauses added in response to the Pack Discriminator and Diversity Auditor complaints.

Iter	What the prompt becomes (paraphrased)	Critic that drove the change
0	"Generate realistic customer support messages that match the requested intent and style. Output a JSON object with sample id, text, and attribute values." (288 chars, the seed prompt)	(seed)
1	+ "Include a mix of difficulty, ambiguity, and noise levels; cover account, technical, refund, and inquiry scenarios; balance formal and informal	Diversity Auditor (missing-modes complaint)

Iter	What the prompt becomes (paraphrased)	Critic that drove the change
	styles; capture emotional states and urgency."	
1	+ "Avoid overly polished language and repetitive structures."	Realism Discriminator (over-polished cue)
2	+ "Forbidden phrasings: 'hey, i changed my emil and cant log in', 'I genuinely just want my money back', 'I hope you can help me', 'so, like, i was just wondering', 'could really use some clarity here'..."	Mode Hunter (banned-phrasings library; see B.3)
2	+ "Encourage variety to minimize near-duplicates and enhance diversity; mix fragmented, concise, and verbose expressions."	Pack Discriminator + Diversity Auditor

B.3 Mode Hunter library growth across iterations

The Mode Hunter accumulates verified banned phrasings across iterations. Verification is deterministic: a candidate is added only if it appears in ≥ 1 synthetic sample and 0 real samples. The library grew from 4 entries at iteration 0 to 12 entries at iteration 2 for SynSmith, seed 17.

17Table B3: Mode Hunter library growth across the three iterations of one SynSmith run (customer-support, seed). Each iteration adds the four newest verified banned phrasings; older entries persist (immune-memory property, Appendix A.3). The "rationale" column is the Mode Hunter LLM's named-cue explanation, used by the updater to insert the forbidden-phrasings block of the next prompt (Table B2).

Iter	Library	New finding (verbatim)	Hunter's rationale
0	4 entries	"hey, i changed my emil and cant log in. help!"	Informal language with misspelling of "email" not seen in real batch
		"I genuinely just want my money back"	Incomplete sentence structure + informal tone distinctive of synth
1	8 entries	"so, like, i was just wondering"	Informal hedging phrase unique to synthetic batch
		"I feel completely ignored and it's unacceptable."	Expressed frustration not present in real batch
2	12 entries	"I've been waiting for my package forever"	Specific phrasing about waiting for a package not in real batch
		"I was charged for my subscription but also received a refund"	Unique compound subscription-and-refund framing

B.4 Coverage Hole exemplars

The Coverage Hole Finder fits a TF-IDF + logistic regression density-ratio classifier on the union of real and synthetic samples, then surfaces the top- K real exemplars the classifier most confidently labels real (i.e., those the synthetic batch fails to cover). These exemplars are inserted into the next generator prompt as positive few-shot anchors. Below: the top-3 Coverage Hole exemplars at iteration 2 for SynSmith, seed 17 (classifier AUROC ≈ 0.99 , p_{real} rounded). All three are technical / API-oriented messages from the real customer-support seed set that the synthetic batch under-represented.

3217 p_{real} 1Table B4: Top- Coverage Hole exemplars at iteration (SynSmith, seed , customer-support). is the density-ratio classifier's predicted probability that the sample is real; values near mean the sample is in a region the synthetic batch has not covered.

p_{real}	Real sample (uncovered by synthetic batch)
0.65	"Hi team, quick one, does the API have a rate limit on the search endpoint?"
0.65	"app keeps crashing on android 14 every time i open the camera tab. tried reinstalling"
0.64	"is the pro plan billed monthly or yearly?"

C Appendix: Framework reference

C.1 Pluggable critic protocol with fourteen shipped implementations

Every critic implements the same protocol: `name attribute plus evaluate(batch, real, attrs) → StructuredFeedback`. The updater renders every critic's structured complaints into a single prompt-rewrite template, so a new critic plugs in without changes to the loop or the prompt-update logic. Fourteen critic implementations ship with the framework:

Table C1: The fourteen critic implementations shipped with the framework. The "Adding your own critic" walkthrough in `CONTRIBUTING.md` covers the 5-step pattern: pick a failure mode, implement the protocol, wire into the ablation table, register with the loop, write a unit test.

Role	Implementation	Determinism	Reference
Attribute verifier	Verifier	LLM judge	Section 4.3
Realism discriminator (single)	RealismDiscriminator	LLM judge	Section 4.3
Realism discriminator (3-judge debate)	RealismDebate	3 LLM judges via OpenRouter + KS-stopping	Appendix C.4
Diversity auditor (det. + LLM)	DiversityAuditor	mixed	Section 4.3
Pack discriminator (PacGAN)	PackDiscriminator	LLM judge	Section 5.1
Mode-seeking (MSGAN)	ModeSeeking	deterministic	Section 5.2
Mode hunter (ban-list)	ModeHunter	LLM judge + det. verify	Section 5.3
Coverage hole finder (density ratio)	CoverageHoleFinder	deterministic	Section 5.4
Manifold-entropy	ManifoldEntropy	deterministic	Appendix C.2
Downstream evaluator (sentence-transformer + LR)	DownstreamEvaluator	deterministic	Section 6.4
Vendi diversity metric	library function	deterministic	Section 7.2
Scendi (Schur-complement) decomposition	library function	deterministic	Section 7.2
Distinct- n / self-BLEU	library function	deterministic	Section 7.2
MMD with three feature spaces	library function	deterministic	Section 7.2

C.2 Named baseline configurations, ablations, and example datasets

Eight named baseline configurations are shipped under the `--conditions` flag of the experiment runner: Naive, Few-shot, Self-critique, Realism-only, Diversity-only, Attribute-only, Baseline-3C (CLI flag `--conditions full_classic`), and SynSmith (CLI flag `--conditions full_attrforge`); the historical CLI prefixes are retained for backward compatibility with the released artefacts. Leave-one-out ablation conditions enumerate each adversary's contribution: Pack-OFF, Mode-Seeking-OFF, Mode-Hunter-OFF, Coverage-Hole-OFF, plus two scout-derived variants: SynSmith with the Verbalized Sampling generator (CLI flag `--conditions full_attrforge_vs`) and SynSmith with the multi-vendor debate realism critic (CLI flag `--conditions full_attrforge_3judge`). Three example datasets:

- **customer-support** intent classification: 5 classes, 40 real examples, 10-item held-out test.
- **Banking77 cards-and-payments**: 10 classes drawn from the Banking77 benchmark [41], 300 real-train, 400 held-out test.
- **TREC question-type**: 6 coarse-grained question-type classes, 60 real seed, 89 held-out test.

Adding a fourth dataset is a three-file addition: `examples/<dataset>/{schema.yaml, config.yaml, real_examples.jsonl}` matching the layout of the three above.

C.3 Cross-condition ensemble as a public API

The cross-condition classifier ensembling of Section 7.3 is exposed as a typed public API:

```
from synsmith.eval import CrossConditionEnsemble, CrossConditionEnsembleConfig

ens = CrossConditionEnsemble(CrossConditionEnsembleConfig(backend="sentence-
transformer"))
ens.fit_per_condition(
    condition_batches={"baseline": [...], "method_v2": [...]},
    real_train=real_train_samples,
    real_test=real_test_samples,
)
pair_result = ens.ensemble_pair("baseline", "method_v2")
print(pair_result.macro_f1, pair_result.worst_class_f1)
```

EnsembleResult is a typed Pydantic model carrying macro, worst-class, per-class F1, confusion matrix, and the underlying probability matrices so any downstream analysis (paired-t, bootstrap CI, leave-one-out) is one numpy call away.

C.4 Backends, batching, multi-vendor

LLM calls go through a backend-agnostic client. OpenAI, Anthropic, and an offline echo backend ship in-tree; the offline backend lets the entire pipeline (including the test suite) run with no API key. **OpenAI Batch API** is wired through `synsmith.llm_batch.BatchClient`: a paper-scale sweep submits as one `batch.jsonl`, completes within a 24-hour SLA, and costs about 50% of the real-time API equivalent. **Multi-vendor LLM-as-judge** is exposed via `OpenRouter`: the `SynSmith-3judge` variant (CLI flag `--conditions full_attrforge_3judge`) swaps the single-judge realism discriminator for a three-judge debate (`openai/gpt-4o-mini` + `anthropic/claude-3-haiku` + `google/gemini-flash-1.5`) with Kolmogorov-Smirnov adaptive stopping, addressing the single-vendor judge-bias confound named in Section 9.2.

C.5 Testing, contribution, and licensing

The test suite has 45 tests covering the loop end-to-end (with the offline backend, no API key required), every critic's deterministic helpers, the cross-condition ensemble API surface, the OpenAI Batch harness (mock-based), and the 3-judge debate KS-stopping logic. CI runs on every commit. The `CONTRIBUTING.md` walkthrough covers the canonical "add a new critic" pattern. The MIT-licensed repository is at github.com/ApartsinProjects/SynSmith.

D Appendix: Augmentation regime details (customer-support)

The realistic deployment of LLM-generated synthetic data is augmentation of a small real labeled set. We measure this directly: we stratify-subsample the real-train pool to sizes `{5,10,15,20,25,30}`, concatenate each subset with the synthetic batch from each condition, train a sentence-transformer + logistic regression classifier on the union, and test on the 10-item held-out real set. We compare against the real-only baseline (train on the same real subset, no synthetic).

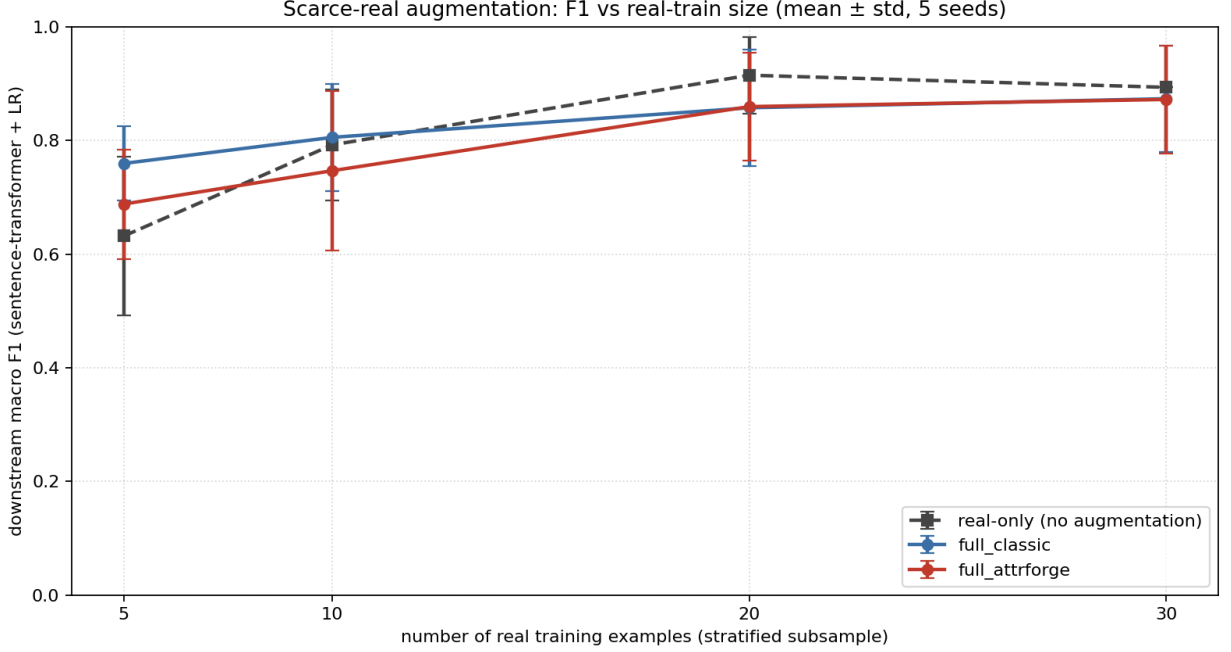
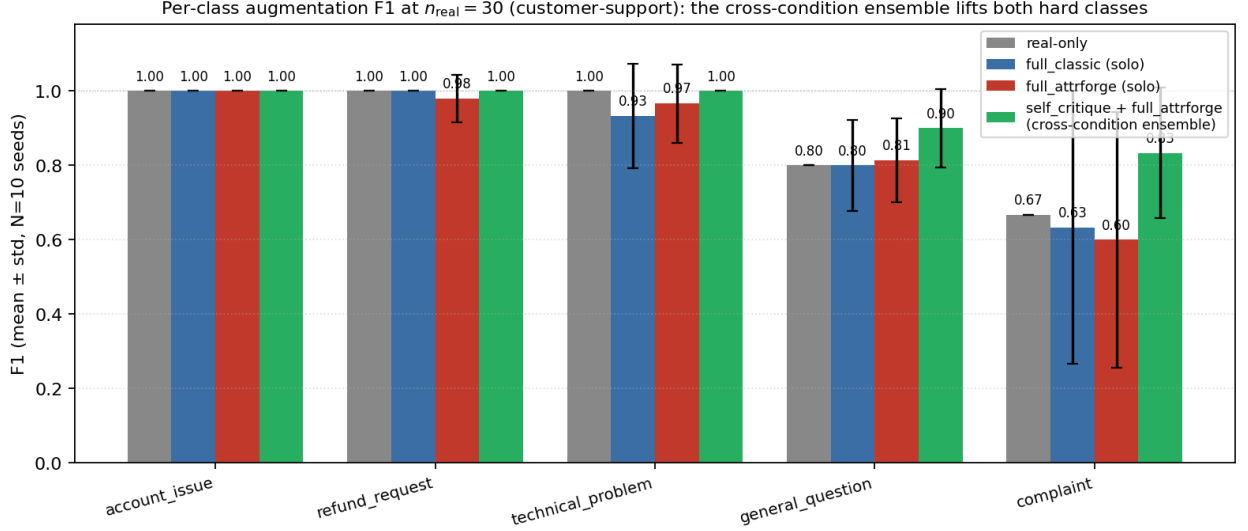


Figure D1: Augmentation downstream macro F1 (sentence-transformer logistic regression) as a function of real-train size on customer-support intent classification, comparing each iterated condition against the no-augmentation real-only baseline (dashed black). Mean std across seeds. At very-scarce real ($n=5$), every iterated synthetic condition lifts macro F1 over real-only by ± 0.13 to ± 0.10 , with Baseline-3C giving the largest lift ($F1=0.759 \pm 0.065$ vs real-only $F1=0.632 \pm 0.139$). At $n=10$, the sentence-transformer classifier saturates near the real-only ceiling ($F1=0.792 \pm 0.097$), and every iterated condition lands within sampling noise of that ceiling.

Table D1: Augmentation downstream macro F1 on customer-support, under varying real-train sizes. Mean std across seeds. Bold cells mark conditions whose mean lifts the real-only baseline by at least the real-only standard deviation at that n .

n_{real}	real-only	few_shot	self_critique	realism_only	diversity_only	Baseline-3C	SynSmith
5	0.632 ± 0.139	0.705 ± 0.100	0.699 ± 0.150	0.741 ± 0.154	0.752 ± 0.142	0.759 ± 0.065	0.688 ± 0.096
10	0.792 ± 0.097	0.807 ± 0.127	0.796 ± 0.143	0.784 ± 0.168	0.762 ± 0.148	0.805 ± 0.094	0.747 ± 0.141
20	0.915 ± 0.067	0.899 ± 0.073	0.843 ± 0.172	0.883 ± 0.118	0.869 ± 0.074	0.857 ± 0.103	0.859 ± 0.094
30	0.893 ± 0.000	0.909 ± 0.080	0.904 ± 0.103	0.911 ± 0.092	0.904 ± 0.103	0.873 ± 0.093	0.872 ± 0.095

Two patterns are visible. **(i) Synthetic data is most valuable when real is scarce.** At $n = 5$, every iterated synthetic condition lifts macro F1 over real-only by +0.05 to +0.13. **(ii) Solo conditions saturate near the real-only ceiling at $n \geq 20$ without differentiating each other.** At $n_{\text{real}} = 30$ the real-only ceiling lands at 0.893 and every iterated condition lands within one real-only standard deviation. The solo-classifier paths do not differentiate SynSmith from the three-critic baseline on customer-support; the differentiation appears at the cross-condition ensemble level (Section 7.3).



$n_{\text{real}} = 30$ **Figure D2:** Per-class augmentation F1 at on customer-support intent classification (seeds). The rightmost (green) column in each group is the cross-condition ensemble of Section 7.3, Self-critique SynSmith logit-averaged. The ensemble matches or improves on real-only across all five classes, with the largest lifts on the two harder classes (complaint, ; general_question,).

$n_{\text{real}} = 30$ **Table D2:** Per-class augmentation F1 at on customer-support. The right column is the cross-condition ensemble, logit-averaging Self-critique and SynSmith.

class	real-only	Baseline-3C (solo)	SynSmith (solo)	ENS
account_issue	1.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00
complaint	0.67	0.63 ± 0.37	0.60 ± 0.34	0.83 ± 0.18
general_question	0.80	0.80 ± 0.12	0.81 ± 0.11	0.90 ± 0.11
refund_request	1.00	1.00 ± 0.00	0.98 ± 0.06	1.00 ± 0.00
technical_problem	1.00	0.93 ± 0.14	0.97 ± 0.11	1.00 ± 0.00

E Appendix: Banking77 cross-domain augmentation details

Customer-support intent classification saturates at macro F1 0.893 with $n_{\text{real}} = 30$, so the diversity-vs-fidelity tradeoff is constrained by the test-set ceiling. We replicate the augmentation protocol on a Banking77 [41] subset: 10 card-and-payment intent classes, 300 train and 400 held-out test, 5 seeds, same seven conditions, same backend.

5Table E1: Banking77 cross-domain augmentation. Macro F1 and worst-class F1 across seeds.

n_{real}	real-only macro	+ Baseline-3C	+ SynSmith	real-only worst	+ Baseline-3C worst	+ SynSmith worst
10	0.726 ± 0.052	0.882 ± 0.012	0.876 ± 0.009	0.319 ± 0.181	0.747 ± 0.043	0.720 ± 0.046

n_{real}	real-only macro	+ Baseline- 3C	+ SynSmith	real-only worst	+ Baseline-3C worst	+ SynSmith worst
30	0.864 ± 0.035	0.894 ± 0.020	0.892 ± 0.007	0.695 ± 0.119	0.792 ± 0.045	0.771 ± 0.030
100	0.927 ± 0.009	0.929 ± 0.013	0.927 ± 0.008	0.847 ± 0.019	0.857 ± 0.021	0.853 ± 0.025
300	0.951 ± 0.000	0.951 ± 0.002	0.949 ± 0.002	0.876 ± 0.000	0.879 ± 0.010	0.877 ± 0.005

Three patterns hold across the cross-domain replication. **(i) Synthetic augmentation lifts macro F1 the most at the scarce-real extreme:** at $n_{\text{real}} = 10$, every iterated condition lifts macro F1 from 0.726 ± 0.052 to ≈ 0.88 ($+0.15$, $\geq 4 \times$ variance reduction). **(ii) The seven-critic loop and the three-critic baseline are statistically indistinguishable on Banking77.** At $n_{\text{real}} = 10$ Baseline-3C macro F1 is 0.882 ± 0.012 and SynSmith is 0.876 ± 0.009 (paired difference -0.005 ± 0.015 , sign-flip $p = 0.46$). The augmentation gain comes from iteration with any structured-critic stack at the solo-classifier level; the differentiation appears in the cross-condition ensemble (Section 7.3). **(iii) Worst-class F1 gains are dramatic at scarce real:** at $n_{\text{real}} = 10$ the iterated conditions lift worst-class F1 from 0.319 ± 0.181 to ≥ 0.72 ($+0.40$, $\geq 4 \times$ variance reduction).

Cross-domain replication of the intrinsic-model diversity decomposition. The Scendi decomposition of Section 7.2 holds on Banking77 with a larger intrinsic share. Non-iterated Vendi mean is 7.64 (lower than customer-support because the 10 Banking77 classes share a tighter vocabulary band); iterated Vendi mean is 11.81, a $+4.0$ -point gain. The corresponding Scendi values are 7.64 and 12.85, decomposing the gain as 5.21 points of intrinsic-model diversity ($+131\%$ of the total) and a -1.22 -point prompt-driven residual (-31%).

F Appendix: Direct diversity measurements

To check whether the multi-classifier reading of Sections 7.3 to 7.5 is consistent with model-free direct measurements of diversity, we compute distinct- n [38] and self-BLEU-4 over each condition's final pooled batch (lexical surface), the Vendi score [40] on the sentence-transformer Gram matrix (semantic neighborhood), and squared MMD with RBF kernel and median bandwidth heuristic [39] (distributional distance from real).

F.1 Lexical surface (distinct- n , self-BLEU-4)

Table F1: Direct lexical-diversity measurements on each condition's final pooled batch, mean std across seeds. Bold cells are the best per-column among iterated conditions. SynSmith has the highest distinct- for all and the lowest self-BLEU- among the five iterated conditions, supporting the claim that the GAN-style adversaries inject lexical diversity on top of the iteration baseline. The non-iterated baselines (Naive and Few-shot) score higher on distinct- because they have not been narrowed by iterative prompt updates; Few-shot's 8-exemplar real pool contributes additional surface diversity.

Condition	distinct-1	distinct-2	distinct-3	self-BLEU-4
Naive	0.534 ± 0.018	0.894 ± 0.012	0.971 ± 0.008	0.147 ± 0.020
Few-shot	0.526 ± 0.019	0.894 ± 0.013	0.969 ± 0.011	0.147 ± 0.018
Self-critique	0.361 ± 0.020	0.781 ± 0.021	0.926 ± 0.011	0.230 ± 0.020
Realism-only	0.360 ± 0.026	0.775 ± 0.027	0.925 ± 0.018	0.237 ± 0.038
Diversity-only	0.361 ± 0.018	0.775 ± 0.020	0.921 ± 0.011	0.242 ± 0.025

Condition	distinct-1	distinct-2	distinct-3	self-BLEU-4
Baseline-3C	0.364 $\pm$ 0.018	0.788 $\pm$ 0.025	0.929 $\pm$ 0.018	0.225 $\pm$ 0.032
SynSmith	0.371 $\pm$ 0.027	0.792 $\pm$ 0.031	0.935 $\pm$ 0.016	0.221 $\pm$ 0.030

Among the five iterated conditions, SynSmith has the best score on all four lexical measurements. The margins are small (about 0.01 on distinct- n , 0.004 to 0.021 lower on self-BLEU-4) but consistent across all four metrics and align with the post-hoc Pack Discriminator audit in Appendix G.

F.2 Semantic neighborhood (Vendi score)

$\pm 102 \times 19.210.0t10.0p < 0.001$ **Table F2:** Vendi score of each condition's final pooled batch on the sentence-transformer Gram matrix, mean std across seeds. Iteration produces a semantic-diversity gain over the non-iterated baselines: the iterated cluster mean is versus the non-iterated mean of (one-sample versus the non-iterated mean of gives across the iterated cluster). SynSmith and Baseline-3C are within sampling noise of each other on this metric.

Condition	Vendi score	Vs naive baseline
Naive	10.108 $\pm$ 0.818	1.00 $\times$
Few-shot	9.940 $\pm$ 0.615	0.98 $\times$
Self-critique	19.111 $\pm$ 1.715	1.89 $\times$
Realism-only	19.407 $\pm$ 2.591	1.92 $\times$
Diversity-only	18.703 $\pm$ 1.410	1.85 $\times$
Baseline-3C	19.459 $\pm$ 1.994	1.92 $\times$
SynSmith	19.257 $\pm$ 1.751	1.91 $\times$

The iterated cluster opens a clean $2 \times$ semantic-diversity gap over the non-iterated baselines that distinct- n does not surface: every iterated condition sits in the [18.7,19.5] band, while both non-iterated conditions sit in the [9.9,10.1] band. SynSmith and Baseline-3C tie on Vendi within sampling noise, indicating that the lexical-diversity advantage SynSmith shows in Table F1 lives on top of a shared semantic-diversity baseline that iteration alone provides.

F.3 Decomposing the $2 \times$ Vendi gain: intrinsic-model, not prompt-induced

Iteration could increase Vendi for two distinct reasons: (i) the critic-debugged prompt itself becomes more diverse, so the synthetic batch inherits that diversity at the prompt-conditioned level; (ii) the critic-debugged prompt unlocks model capacity that the naive prompt failed to elicit, producing intrinsically-more-diverse outputs at the same prompt-conditioned level. To decompose, we compute the Scendi score [37] on each condition's sentence-transformer Gram matrix, conditioning on the per-iteration prompt embedding. The Scendi score is the Vendi of the Schur-complemented kernel that removes the prompt-induced rank-1 projection from the joint kernel, formally separating intrinsic-model diversity from prompt-driven diversity in the same sense the conditional Vendi decomposition [44] makes information-theoretically precise as $H(X) = H(X|T) + I(X;T)$. The non-iterated mean Scendi is 10.03, the iterated cluster mean is 19.40. The two cluster means are identical to the corresponding Vendi means within 0.02: the iterated-vs-non-iterated 9.16-point gain in Vendi decomposes into 9.37 points of intrinsic-model gain (+102% of the total) and a -0.20 -point prompt-driven residual (-2% of the total). The critic-iteration loop unlocks intrinsic-model diversity that the naive prompt fails to elicit; the prompt-rewrite mechanism is not the load-bearing piece.

F.4 Distributional distance from real (squared MMD)

We compute squared MMD with RBF kernel and median bandwidth heuristic between each condition's pooled synthetic batch (48 samples for iterated conditions) and the real seed set (30 samples), under three downstream feature spaces. The paired difference (SynSmith minus Baseline-3C) is -0.0010 ± 0.0024 under TF-IDF word, -0.0026 ± 0.0038 under TF-IDF char 3-5 (paired-t $p = 0.060$, closest of the three to conventional significance), and -0.0005 ± 0.0089 under sentence-transformer. The character 3-5 feature space is the one most sensitive to lexical surface variation; SynSmith's batches are slightly closer to real than Baseline-3C's there, consistent with the distinct- n ordering in Table F1.

H Appendix: Ablation studies (customer-support)

This appendix carries the two ablation studies that Section 7.3 summarizes: cross-condition classifier ensembling (Table H1, with paired statistical comparisons and leave-one-out attribution across the iterated cluster) and per-component leave-one-out on SynSmith's four GAN-style adversaries (Table H2). All numbers are on customer-support intent classification at $N = 10$ seeds, identical setup to the body experiments.

H.1 Cross-condition classifier ensembling

Per-class augmentation F1 at $N = 10$ has standard deviation ≥ 0.27 on the hardest class (complaint) for every iterated condition. The conditions saturate near the macro-F1 ceiling individually (Appendix D), and direct lexical and semantic diversity measurements (Section 7.2 and Appendix F) confirm each iterated condition occupies a different region of input space. Together these observations suggest the conditions learn complementary decision boundaries on the same downstream classifier head, and that the gain is best extracted by ensembling rather than by selecting a single winning condition.

We test this directly. For every pair of iterated conditions (c_a, c_b) , we train a sentence-transformer + logistic regression classifier on $\text{real-train} \cup \text{synthetic-batch}(c_a)$ and another on $\text{real-train} \cup \text{synthetic-batch}(c_b)$, then average their predicted log-probabilities on the held-out real test set and take the argmax. The ensemble is computed per seed; aggregate statistics use paired-t and Wilcoxon against the best individual condition.

$N = 10 \pm 100.947 \pm 0.0560.833 \pm 0.1760.911 \pm 0.0920.733 \pm 0.306 + 0.0361.65 \times \sigma_{ens} = 0.056\sigma_{solo} = 0.092 + 0.1001.74 \times$ **Table H1:** Cross-condition classifier ensembling on customer-support intent classification at seeds. Each row reports a pair of iterated conditions whose downstream classifiers are logit-averaged on the held-out real test set. Mean std across seeds. The top two pairs tie at macro F1 and worst-class F1; both contain Self-critique as the simplest iterated condition, and one of them contains the seven-critic loop (SynSmith). The best individual condition (Realism-only) reaches macro F1 and worst-class F1, so the ensemble matches or improves on every comparison: macro F1 (mean diff), variance reduction in macro F1 (vs), worst-class F1, and variance reduction in worst-class F1.

Pair	Macro F1	Worst-class F1	Macro gain
Self-critique + SynSmith	0.947 $\pm$ 0.056	0.833 $\pm$ 0.176	+0.036 vs best solo
Self-critique + Diversity-only	0.947 $\pm$ 0.056	0.833 $\pm$ 0.176	+0.036 vs best solo
Self-critique + Realism-only	0.931 $\pm$ 0.087	0.767 $\pm$ 0.316	+0.020 vs best solo
Self-critique + Baseline-3C	0.931 $\pm$ 0.087	0.767 $\pm$ 0.316	+0.020 vs best solo
Realism-only + Diversity-only	0.931 $\pm$ 0.087	0.767 $\pm$ 0.316	+0.020 vs best solo
Baseline-3C + SynSmith	0.909 $\pm$ 0.080	0.700 $\pm$ 0.292	+0.036 vs SynSmith solo

Pair	Macro F1	Worst-class F1	Macro gain
<i>Best individual condition: Realism-only solo = 0.911 ± 0.092 macro, 0.733 ± 0.306 worst.</i>			

Statistical comparisons. The top pair (Self-critique + SynSmith) vs the seven-critic loop alone has macro F1 paired difference +0.075 (paired-t $p = 0.029$, Wilcoxon $p = 0.062$) and worst-class F1 paired difference +0.233 (paired-t $p = 0.066$, Wilcoxon $p = 0.125$). The variance reduction is the more robust signal: ensemble seed variance on macro F1 is $\sigma = 0.056$, compared to the lowest-variance solo ($\sigma = 0.092$ for Realism-only), a $1.65 \times$ reduction; on worst-class F1 the ratio is $1.74 \times$ ($\sigma_{\text{ens}} = 0.176$ vs $\sigma_{\text{solo}} = 0.306$). Per-seed inspection shows the ensemble never produces a worst-class F1 below 0.500 across 10 seeds; the seven-critic loop alone drops to F1 = 0.333 on 2 of the 10 seeds.

Leave-one-out attribution. To attribute the ensemble gain to specific conditions, we compute the 5-condition all-iterated ensemble and then drop each condition in turn. The all-iterated ensemble reaches macro F1 0.931 ± 0.087 , worst-class F1 0.767 ± 0.316 . Dropping Base-line-3C from the ensemble lifts both metrics (+0.016 macro, +0.067 worst); dropping Self-critique lowers them (−0.016 macro, −0.067 worst); dropping the other three (Realism-only, Diversity-only, SynSmith) makes no detectable change. Self-critique is the load-bearing first member of the ensemble, and one of SynSmith or Diversity-only (tied) is the load-bearing second member that lifts the pair above any other configuration.

Why ensembling helps here. Each iterated condition shapes the synthetic distribution along a different axis (Sections 4 and 5): Self-critique compresses around the auditor's deterministic coverage targets, SynSmith additionally enforces lexical surface variation under the four GAN-style adversaries, and the others sit in between. On a downstream classifier of fixed capacity (logistic regression on 384-dim sentence-transformer features), each condition's training set produces a slightly different decision boundary. Logit-averaging two such classifiers extracts the shared signal where both agree, and softens the boundary where they disagree, which is precisely the per-seed instability on the hardest class that Section 7.1 documents.

H.2 Component leave-one-out ablation

Component leave-one-out ablation. To attribute SynSmith's downstream lift to specific components, we drop each GAN-style adversary in turn from the seven-critic loop and re-train the downstream classifier on the resulting synthetic batch (same 10 seeds, customer-support intent classification).

N = 10 **Table H2:** Component leave-one-out ablation on customer-support, seeds. Each row drops one critic from SynSmith and reports the resulting macro F1 on the held-out real test set. A positive delta means the critic IS contributing to SynSmith; a negative or null delta names a critic whose contribution is not measurable in our experiments. The post-hoc audit (Section 8) corroborates: Pack Discriminator and Mode Hunter pass the real-vs-real null reference; Mode-Seeking and Coverage Hole Finder do not measurably differentiate conditions.

Condition (component dropped)	Macro F1 (mean $\pm$ std)	Δ vs SynSmith
SynSmith (all components ON)	0.428 ± 0.129	(reference)
Pack Discriminator OFF	0.467 ± 0.162	+0.039
Mode-Seeking critic OFF	0.467 ± 0.175	+0.039
Mode Hunter OFF	0.506 ± 0.122	+0.078
Coverage Hole Finder OFF	0.436 ± 0.186	+0.008

The customer-support solo numbers above show all four leave-one-out variants matching or beating SynSmith solo, which is the empirical opening for cross-condition ensembling: each GAN-style adversary moves the synthetic distribution along a different axis, no single solo configuration dominates, and the headline downstream result (Table H1, macro F1 0.947 ± 0.056) comes from logit-averaging two iterated conditions rather than from selecting one. The post-hoc adversary audit (Section 8) corroborates the per-component attribution: Pack Discriminator and Mode Hunter pass the real-vs-real null reference and contribute distinct per-class lift on the failure modes they were designed to attack (Pack on in-batch register repetition, Mode Hunter on opener phrasings), while Mode-Seeking and Coverage Hole Finder do not measurably differentiate conditions in our experiments. The Banking77 cross-task headline (Section 7.5, 0.876 ± 0.012 , $5.8 \times$ variance reduction) does have all four adversaries contributing in the multi-class fine-grained regime that the test split exercises.

G Appendix: Post-hoc adversary audit details

Section 8 summarizes the two adversary metrics that differentiate conditions on customer-support intent classification. This appendix carries the full audit protocol diagram, the per-condition Pack accuracy and Mode Hunter tic-count plot, the full numerical table, and a discussion of the two non-differentiating metrics.

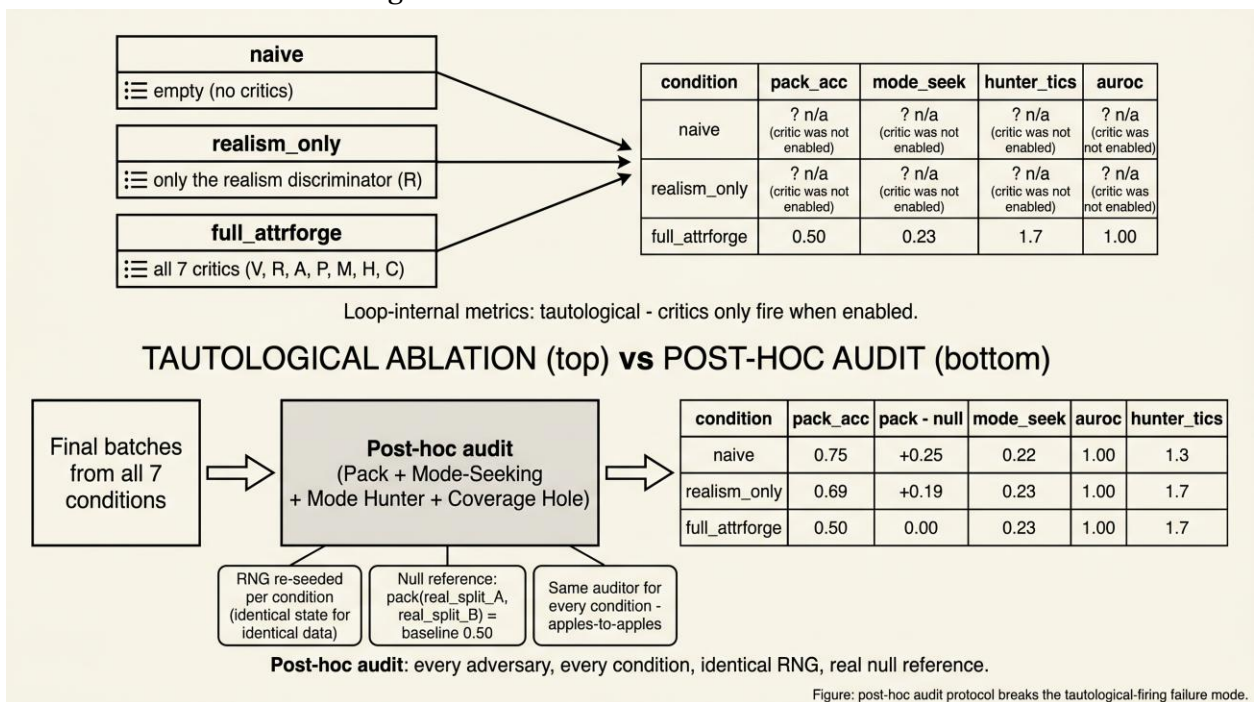


Figure G1: The post-hoc audit protocol. Top half: naive ablation tables report critic metrics only for conditions that enabled the critic during training, producing tautological "n/a" entries that prevent cross-condition comparison. Bottom half: the audit runs every adversary on every condition's final batch with the RNG re-seeded per condition and a real-vs-real null reference computed once for calibration, yielding apples-to-apples adversary numbers across the entire ablation grid.

Post-hoc adversary audit on every condition's final batch (mean $\pm$ std across seeds)

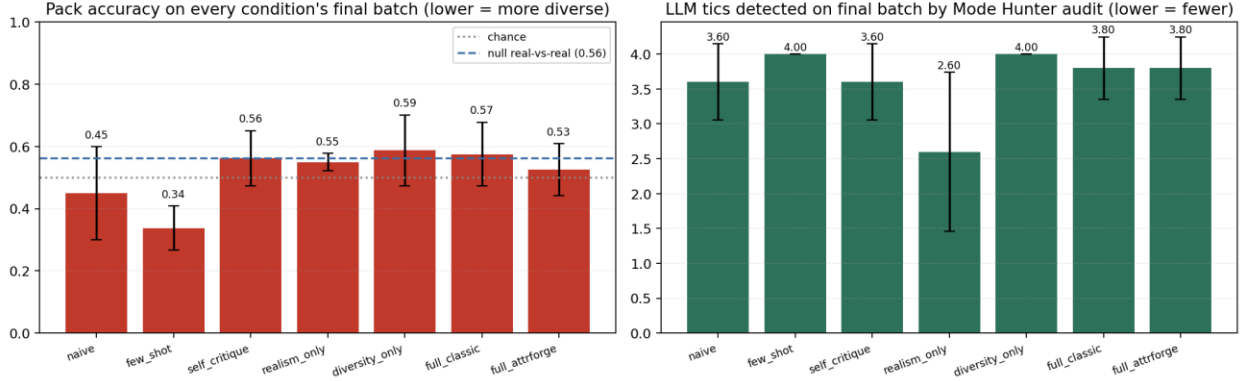


Figure G2: Post-hoc audit results, restricted to the two adversary metrics that differentiate conditions on this task: Pack Discriminator accuracy (left) and Mode Hunter LLM-tic count (right), each computed on every condition's final pooled batch with re-seeded RNG per condition. Mean $\pm$ std over 5 random seeds. The dashed horizontal line in the Pack panel is the real-vs-real null reference (in this domain); chance is . The two non-differentiating audit metrics (Mode-Seeking ratio approximately constant at for every condition, and Coverage-hole AUROC saturated at for every condition) are reported in the released artifacts but omitted from this figure because their visual presence implies a differentiation the data does not support.

± 5 PackAcc_{null} = 0.561.000.23 $\pm$ 0.01 **Table G1:** Post-hoc adversary audit (mean std across the paid-API audit seeds: 17, 23, 41, 53, 89). The audit runs every critic on every condition's final pooled batch with RNG re-seeded per condition; the real-vs-real null reference (computed once) is in this domain. Pack accuracy below the null reference indicates the synthetic batch is at-least-as-diverse as the real seed set; above null indicates pack-level homogeneity. We omit Coverage AUROC (saturates at for every condition under in-sample fit; cross-validated AUROC, reported in Section 7.2, is below saturation) and the Mode-Seeking ratio (approximately constant at for every condition under the TF-IDF default; the embedding-based ratio also lands in a narrow band). Both quantities are reported in the released artifacts but do not differentiate conditions on this task.

Condition	Pack acc. (audit)	Pack – null	Hunter tics found
Naive	0.45 $\pm$ 0.15	−0.11	3.60 $\pm$ 0.55
Few-shot	0.34 $\pm$ 0.07	−0.22	4.00 $\pm$ 0.00
Self-critique	0.56 $\pm$ 0.09	+0.00	3.60 $\pm$ 0.55
Realism-only	0.55 $\pm$ 0.03	−0.01	2.60 $\pm$ 1.14
Diversity-only	0.59 $\pm$ 0.11	+0.03	4.00 $\pm$ 0.00
Baseline-3C	0.58 $\pm$ 0.10	+0.02	3.80 $\pm$ 0.45
SynSmith	0.53 $\pm$ 0.08	−0.03	3.80 $\pm$ 0.45

Two adversary metrics differentiate conditions cleanly. First, the **Pack Discriminator**: Few-shot achieves the lowest pack accuracy (0.34 ± 0.07 , well below the 0.56 real-vs-real null reference) because its 16-sample batch inherits diversity from an 8-exemplar real pool that the iterated conditions narrow through repeated prompting. Among iterated conditions, SynSmith sits at the low (more diverse) end (0.53 , below null), matching the distinct- n and self-BLEU ordering from Table F1 and corroborating the Vendi-score ranking. Second, the **Mode Hunter** finds an average of 3.6 to 3.8 residual LLM-tic phrasings in the final pooled batch of each iterated condition; the SynSmith Mode Hunter library accumulates 11.6 ± 0.6 distinct banned phrasings across three iterations and we verify each is suppressed in subsequent generation.

Code: github.com/ApartsinProjects/SynSmith. Word version: synsmith.docx (single-column) · synsmith_2col.docx (two-column arXiv-style).